

Lecture 8: Logical Agents Part I

Russell and Norvig: Chapter 7

CS-4820/5820

Tu/Th 12:15 PM-1:30 PM

Room: Centennial Hall 106

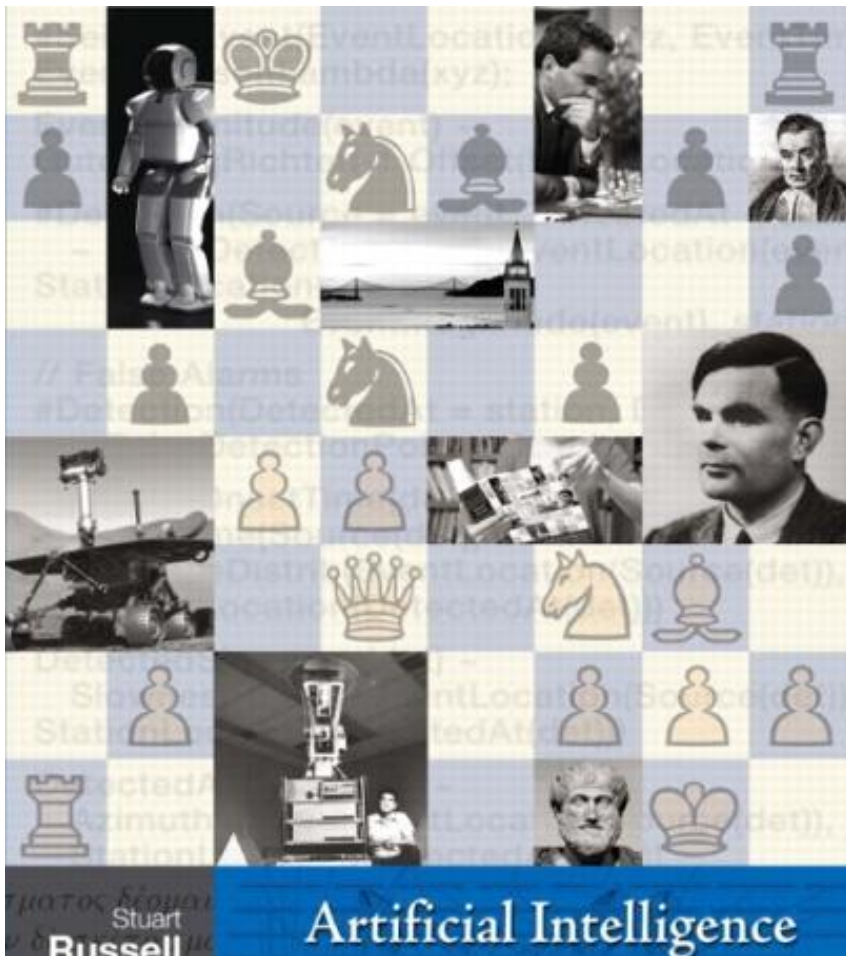
Instructor: Adham Atyabi

Office: Engineering 243

Office Hours: Mon 9:00 AM-14:00 PM.

Email: aatyabi@uccs.edu

Teaching Assistant: Ali Al Shami
(aalshami@uccs.edu)



Outline

- Knowledge-based agents
- Wumpus world
- Logic in general - models and entailment
- Propositional (Boolean) logic
- Equivalence, validity, satisfiability
- Inference rules and theorem proving
 - forward chaining
 - backward chaining
 - resolution

Logical Agent

- What is a logical agent?
- We can tell it facts or ask for inference
- For example
 - You can tell the agent the father of john is Bob
 - You can tell the agent the jane is john's sister
 - You can tell the agent the john's father is the same as john's sister's father
 - You can ask the agent who is jane's father

Knowledge based Agents

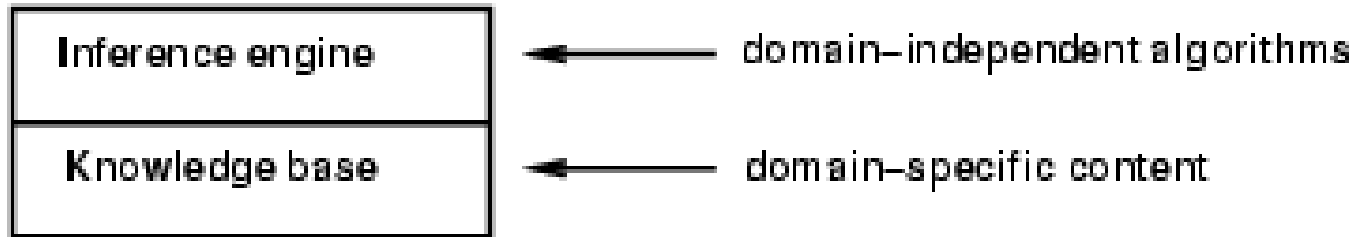
the main components of a knowledge-based system are:

- knowledge base
- Knowledge base representation language
- Inference engine
- have background knowledge about the world

Knowledge based Agents

- At every step:
 - Construct a sentence with assertion about percepts
 - Constructs a sentence asking what action is next
 - Constructs a sentence asserting that action

Knowledge bases



- Knowledge base = set of **sentences** in a **formal** language
- **Declarative** approach to building an agent (or other system):
 - Tell it what it needs to know
- Then it can Ask itself what to do - answers should follow from the Knowledge Base (KB)
- Agents can be viewed at the **knowledge level**
i.e., what they know, regardless of how implemented
- Or at the **implementation level**
 - i.e., data structures in KB and algorithms that manipulate them

A simple knowledge-based agent

```
function KB-AGENT(percept) returns an action  
  static: KB, a knowledge base  
           t, a counter, initially 0, indicating time  
  
  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))  
  action  $\leftarrow$  ASK(KB, MAKE-ACTION-QUERY(t))  
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))  
  t  $\leftarrow$  t + 1  
  return action
```

- The agent must be able to:
 - Represent states, actions, etc.
 - Incorporate new percepts
 - Update internal representations of the world
 - Deduce hidden properties of the world
 - Deduce appropriate actions

Wumpus World PEAS description

- **Performance measure**

- gold +1000, death -1000
- -1 per step, -10 for using the arrow

- **Environment**

- Squares adjacent to Wumpus are smelly
- Squares adjacent to pit are breezy
- Glitter if gold is in the same square
- Shooting kills Wumpus if you are facing it
- Shooting uses up the only arrow
- Grabbing picks up gold if in same square
- Releasing drops the gold in same square

- **Sensors:** Stench, Breeze, Glitter, Bump, Scream

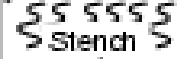
- **Actuators:** Left turn, Right turn, Forward, Grab, Release, Shoot

4

3

2

1

			
	  		
			
 START			

1

2

3

4

Wumpus world characterization

- Fully_Observable No – only **local** perception
- Deterministic Yes – outcomes exactly specified
- Episodic No – sequential at the level of actions
- Static Yes – Wumpus and Pits do not move
- Discrete Yes
- Single-agent? Yes – Wumpus is essentially a natural feature

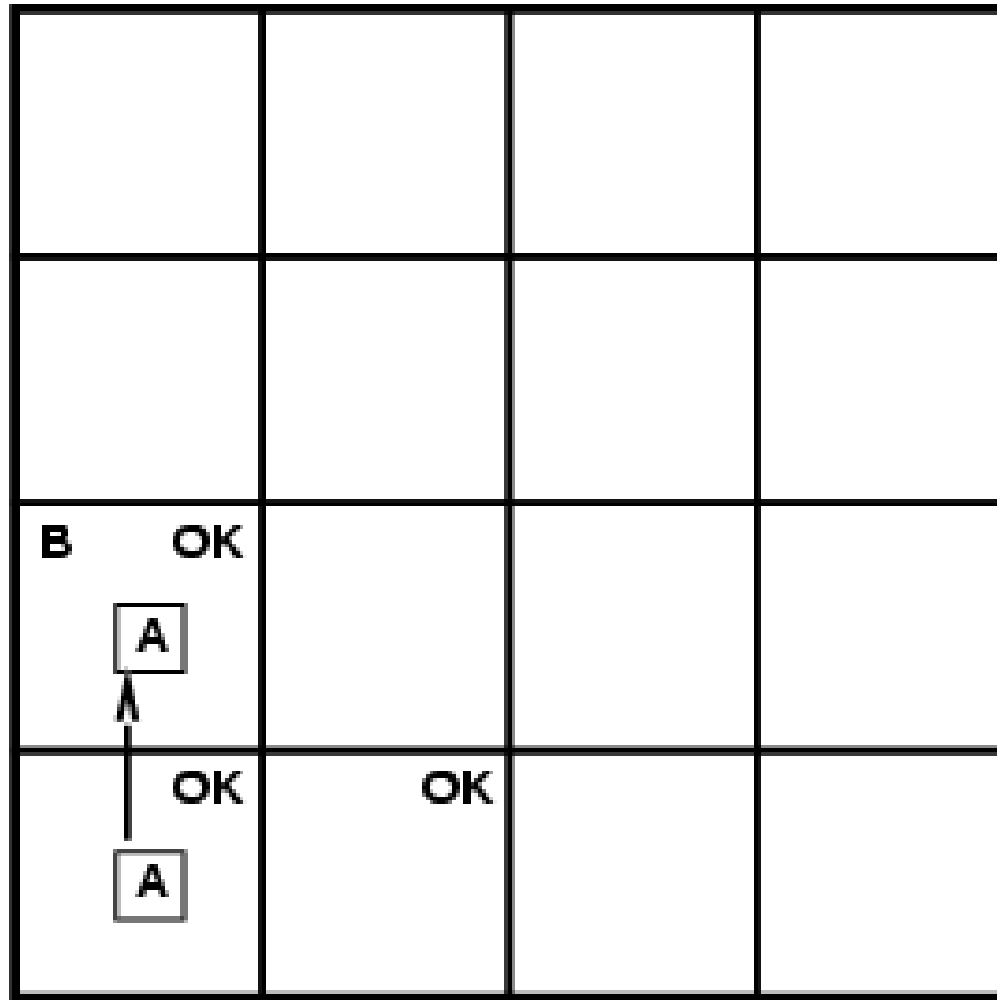
Exploring a wumpus world

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2	3,2	4,2
1,1 A OK	2,1 OK	3,1	4,1

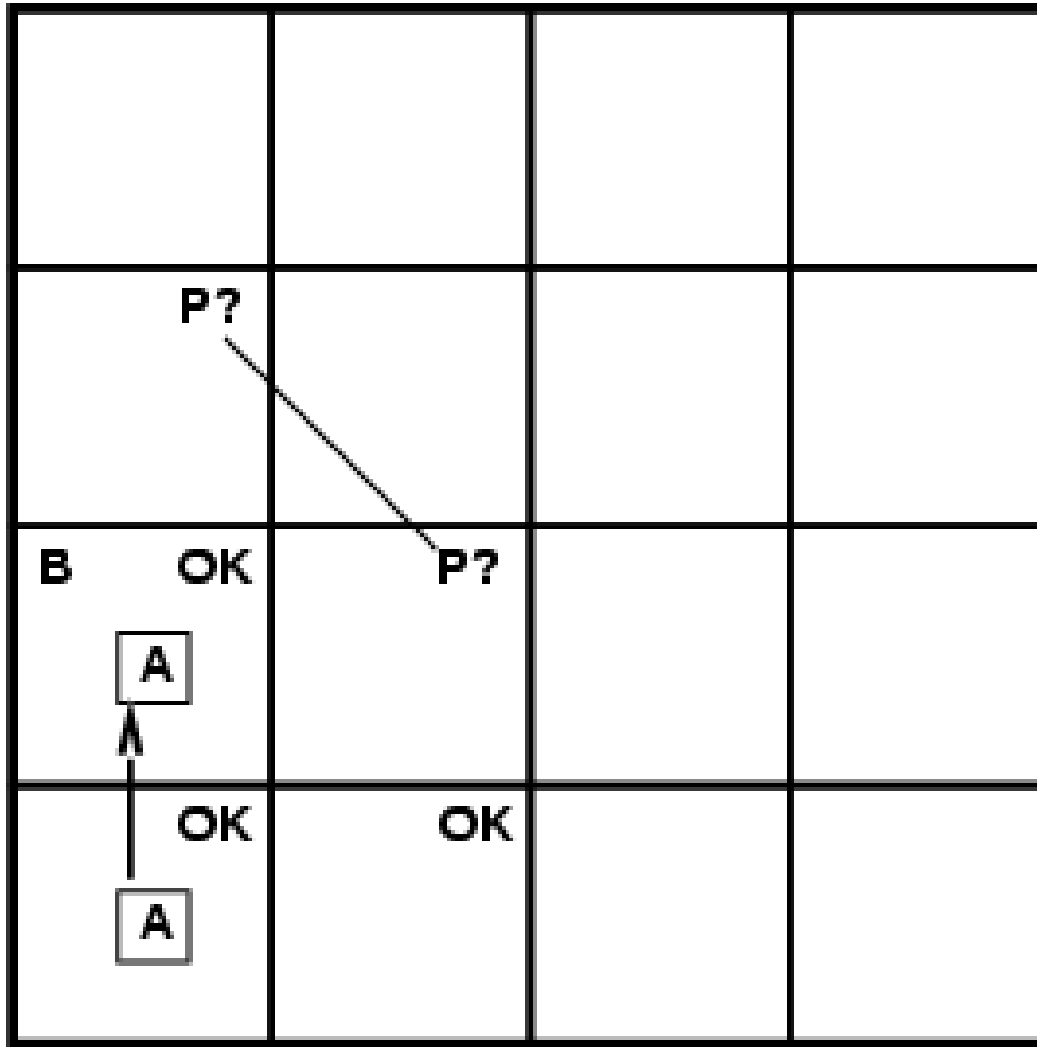
A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

[None, None, None, None, None]

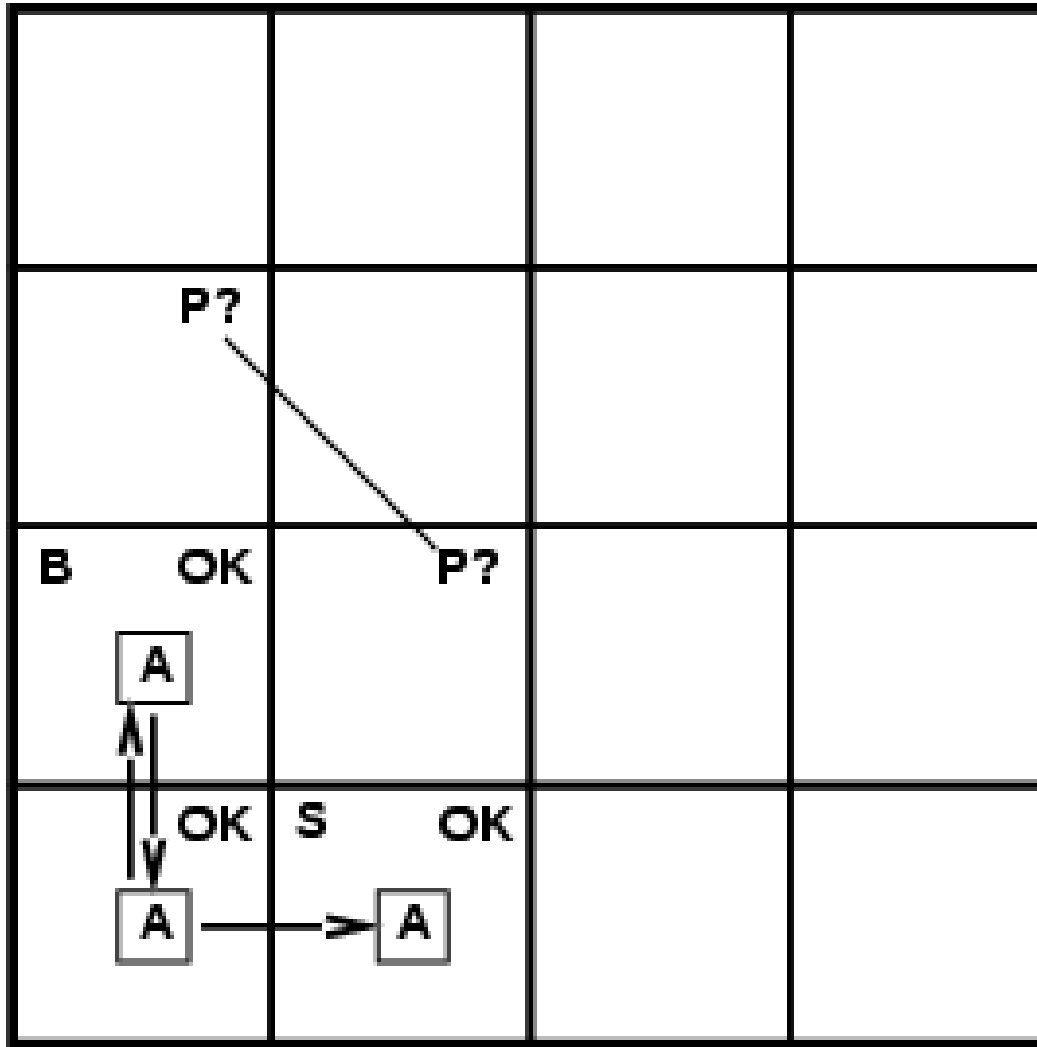
Exploring a wumpus world



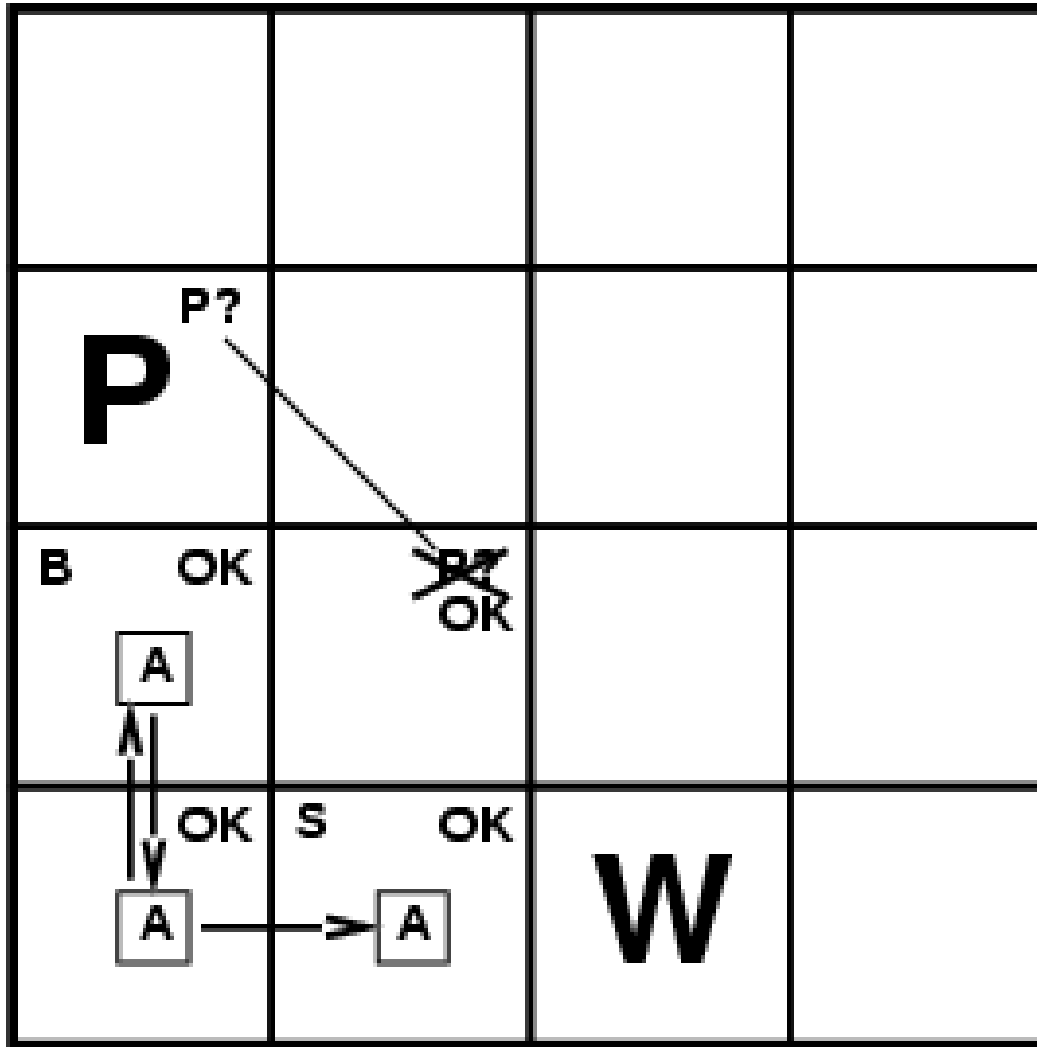
Exploring a wumpus world



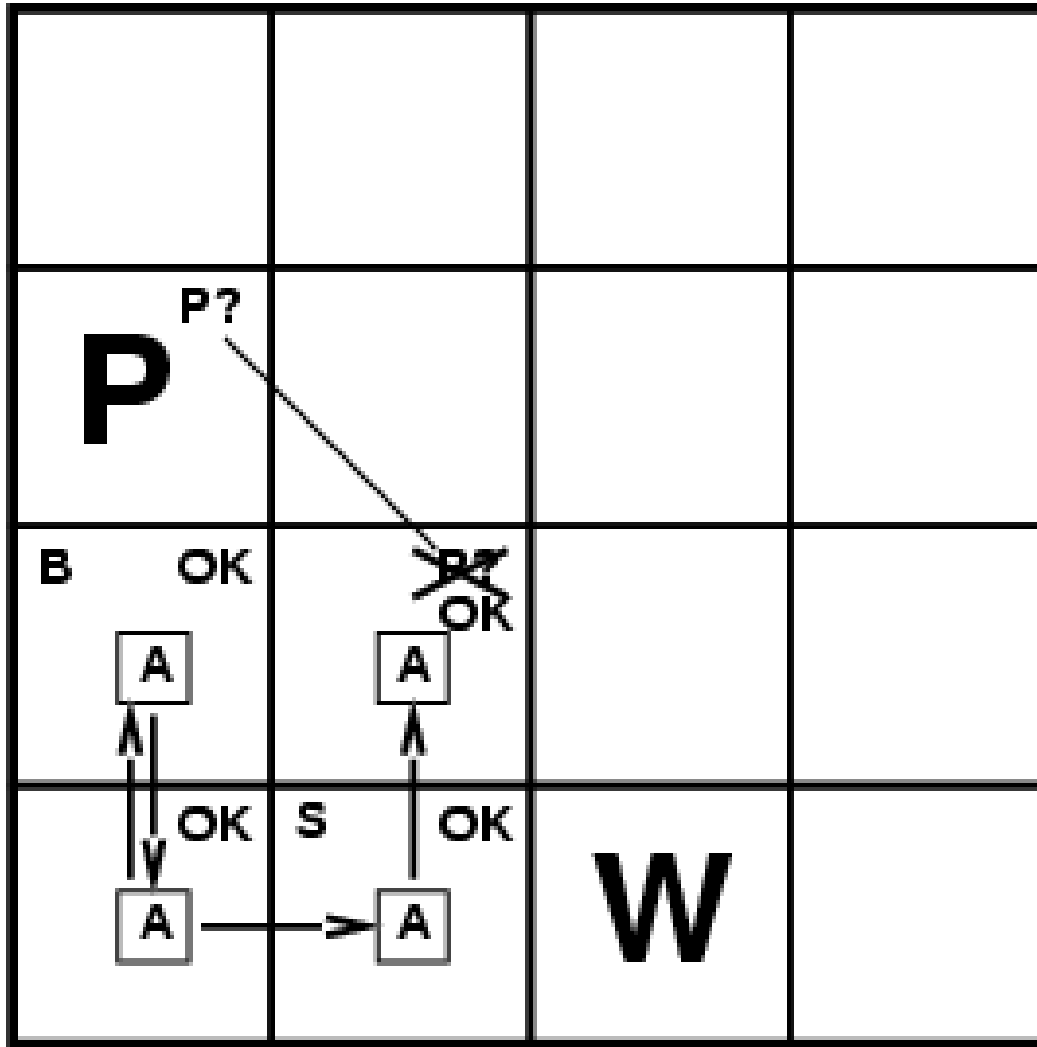
Exploring a wumpus world



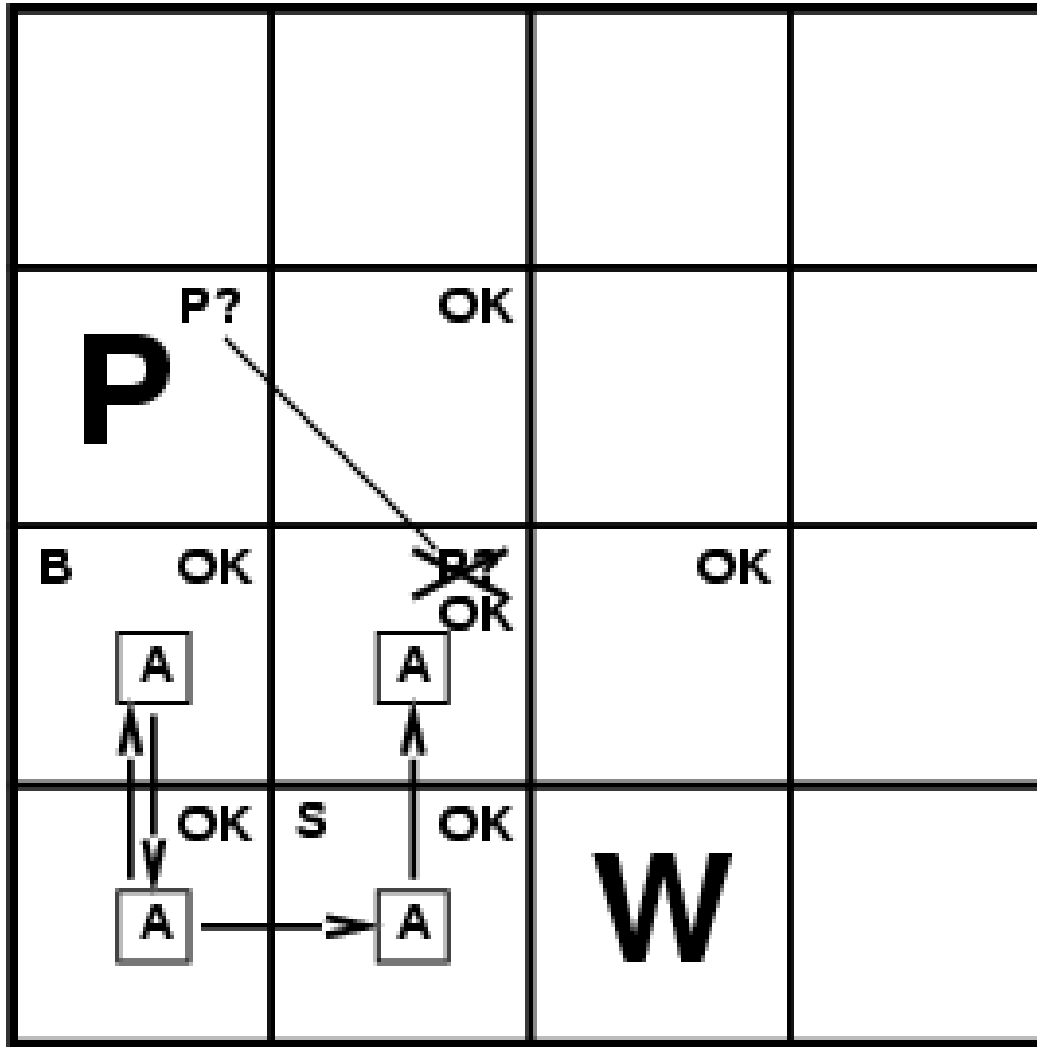
Exploring a wumpus world



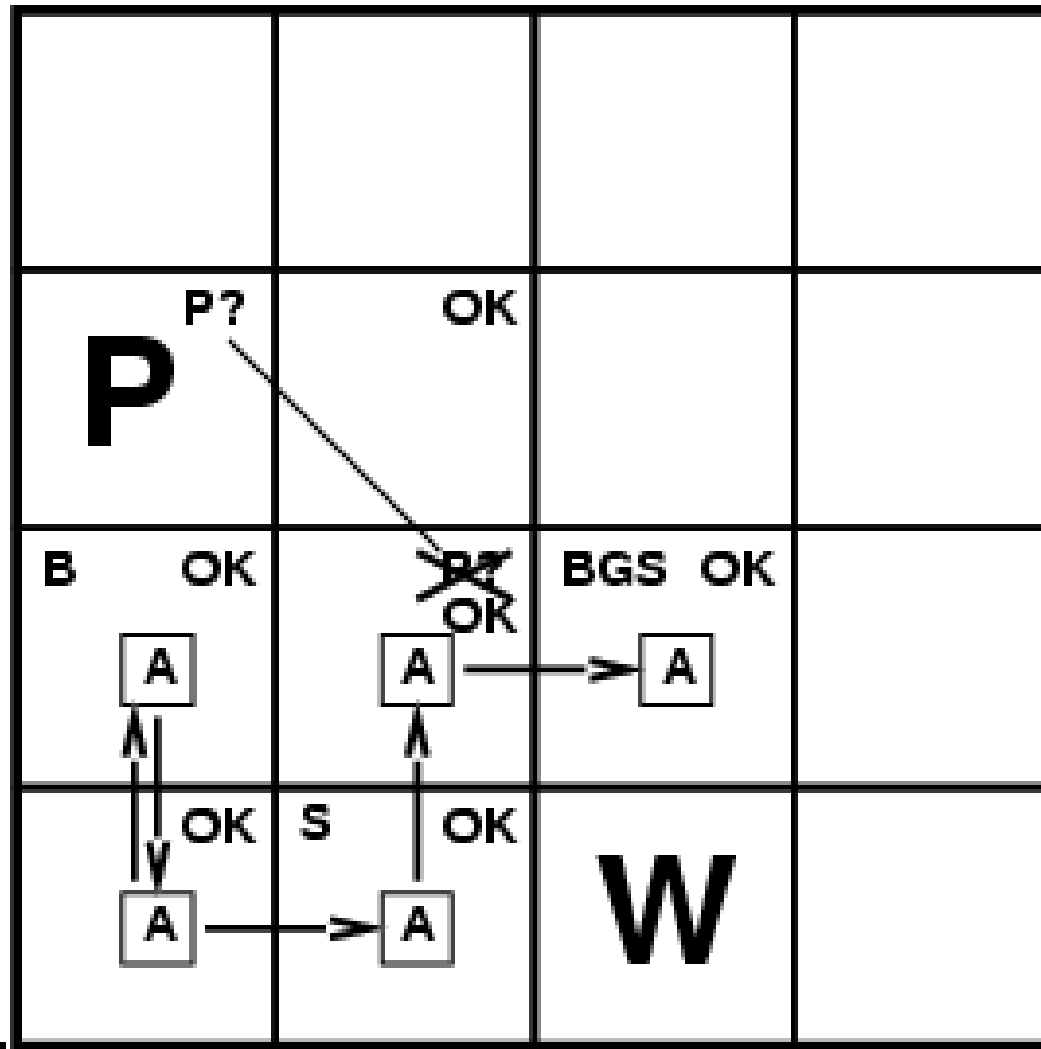
Exploring a wumpus world



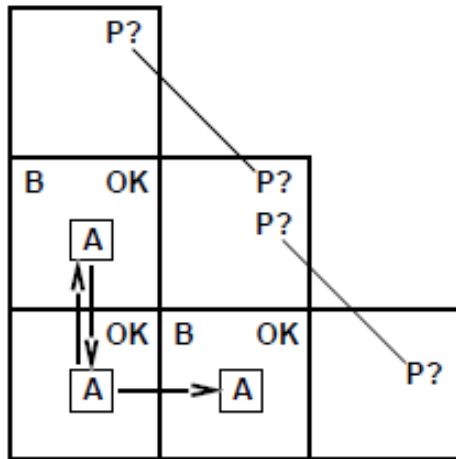
Exploring a wumpus world



Exploring a wumpus world

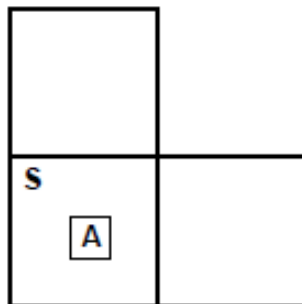


Other tight spots



Breeze in (1,2) and (2,1)
 $\Rightarrow$ no safe actions

Assuming pits uniformly distributed,
 (2,2) has pit w/ prob 0.86, vs. 0.31



Smell in (1,1)

$\Rightarrow$ cannot move

Can use a strategy of coercion:

shoot straight ahead

wumpus was there $\Rightarrow$ dead $\Rightarrow$ safe

wumpus wasn't there $\Rightarrow$ safe

Logic in general

- **Logics** are formal languages for representing information such that conclusions can be drawn
- **Syntax** defines the sentences in the language
 - $x+4=6$
 - $4x=6+$
- **Semantics** define the "meaning" of sentences;
 - i.e., define **truth** of a sentence in a world
- The **models** describe the possible world.
- $M(\alpha)$ is the set of all models of α . If sentence α is **true** for model m , α satisfies m .
- E.g., the language of arithmetic
 - $x+2 \geq y$ is a sentence; $x+2+y > \{ \}$ is not a sentence
 - $x+2 \geq y$ is true if the number $x+2$ is no less than the number y
 - $x+2 \geq y$ is true in a world where $x = 7, y = 1$
 - $x+2 \geq y$ is false in a world where $x = 0, y = 6$

Entailment

- **Entailment** means that one thing **follows from** another:

$$KB \models \alpha$$

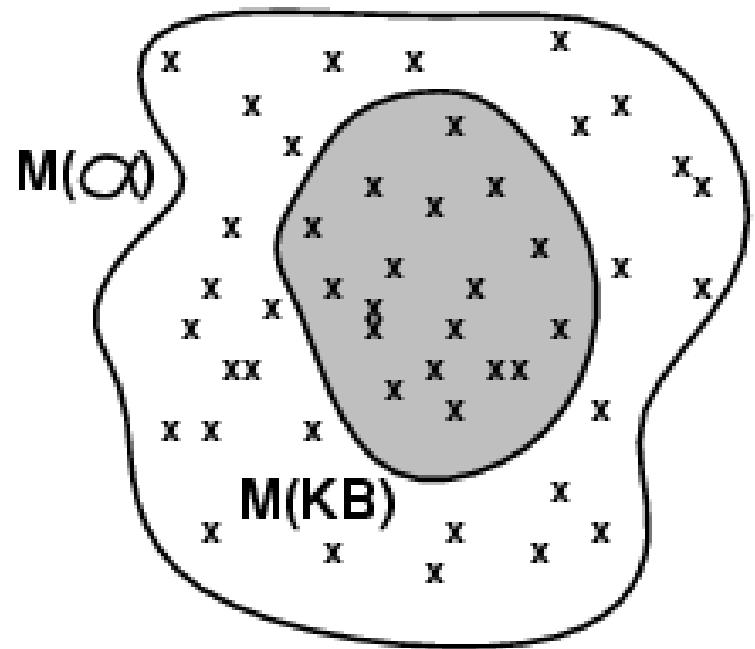
$$KB \models \alpha \text{ iff } M(\alpha) \subseteq M(KB).$$

Knowledge base *KB* entails sentence α if and only if α is true in all worlds where *KB* is true

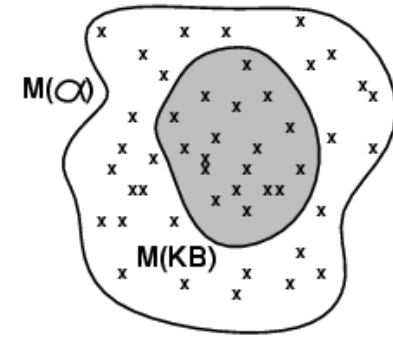
- E.g., the KB containing “the Giants won” and “the Reds won” entails “Either the Giants won or the Reds won”
- E.g., $x+y = 4$ entails $4 = x+y$
- Entailment is a relationship between sentences (i.e., **syntax**) that is based on **semantics**

Models

- Logicians typically think in terms of **models**, which are formally structured worlds with respect to which truth can be evaluated
- We say m **is a model of** a sentence α if α is true in m
- $M(\alpha)$ is the set of all models of α
- Then $KB \models \alpha$ if $M(KB) \subseteq M(\alpha)$
 - $KB = \text{Giants won and Reds won}$
 $\alpha = \text{Giants won}$
 - $(x=0) \models (xy=0)$
 - $(p = \text{true}) \models (p \vee q) = \text{true}$

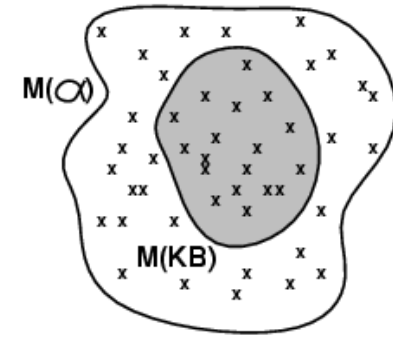


Models



- $KB \models \alpha$: represents a fundamental concept in the realm of logic and knowledge bases, signifying that a knowledge base (KB) entails a statement (α).
- KB is a collection of known facts, assertions, and rules about a domain of interest.
- $\models$ represents a statement, proposition, or assertion that we are checking to see if it can be logically inferred from the knowledge base.
- $KB \models \alpha$ reads as "KB entails α ," or " α is entailed by KB," meaning that whenever the facts and rules in KB are true, then α must also be true. This is a way to say that α is a logical consequence of the knowledge base KB.

Models



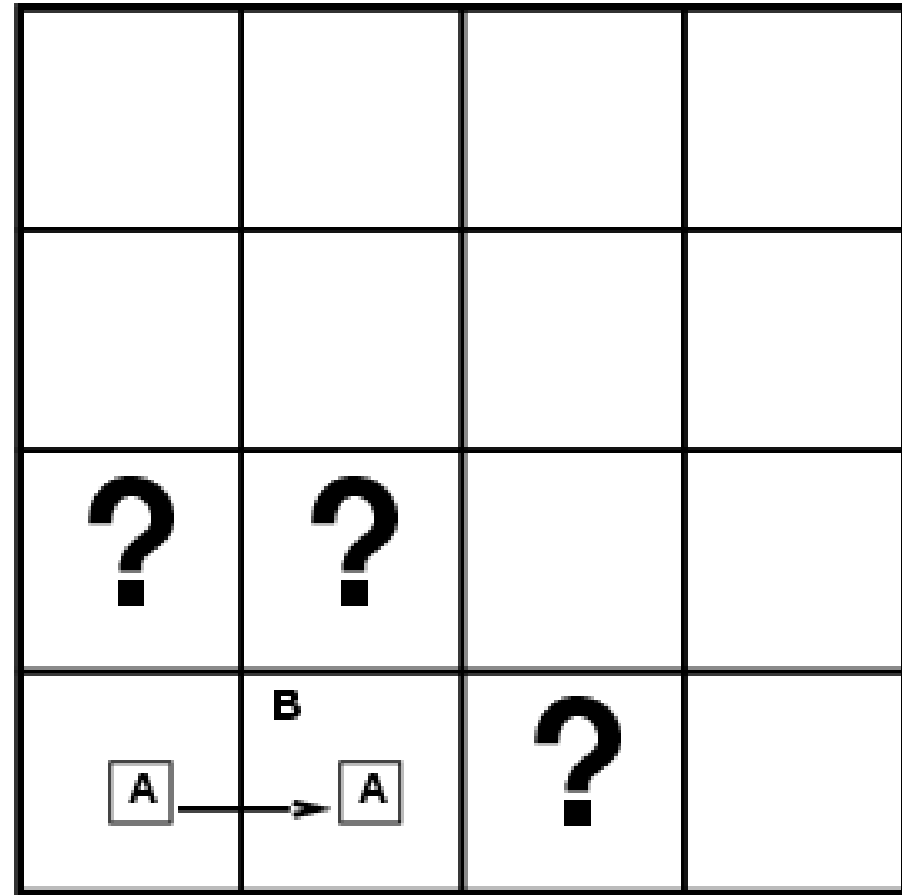
- $KB \models \alpha$ if $M(KB) \subseteq M(\alpha)$
- $M(KB) \subseteq M(\alpha)$
- $M(KB)$: Represents the set of all models (interpretations of the world) where the knowledge base KB holds true. A model is a specific assignment of truth values to statements that makes all statements in KB true.
- $M(\alpha)$: Represents the set of all models where the statement α holds true.
- The expression " $M(KB) \subseteq M(\alpha)$ " means that every model in which the knowledge base KB is true is also a model in which the statement α is true. In simpler terms, **there's no possible interpretation of the world that makes all statements in KB true without also making α true**. This is the model-theoretic way of expressing that the knowledge base KB entails the statement α .
- " $KB \models \alpha$ iff $M(KB) \subseteq M(\alpha)$ " expresses the concept of entailment both logically and model-theoretically.
- α is a logical consequence of a knowledge base KB if and only if every model (interpretation) that makes KB true also makes α true.

Entailment in the wumpus world

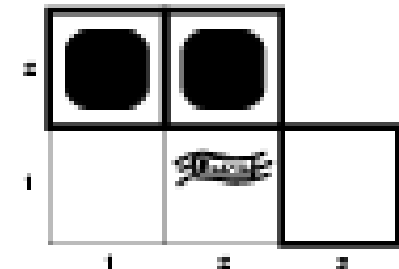
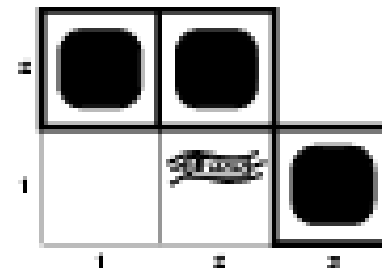
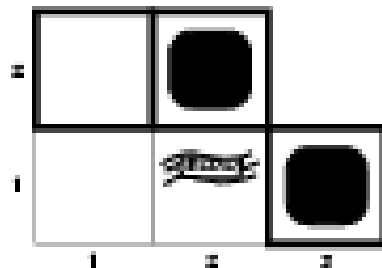
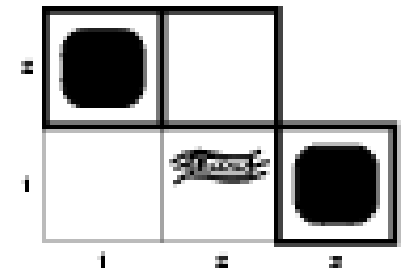
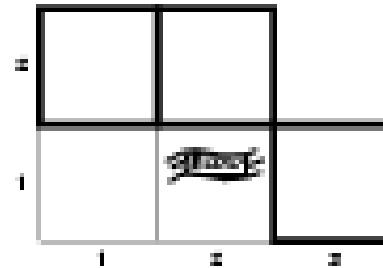
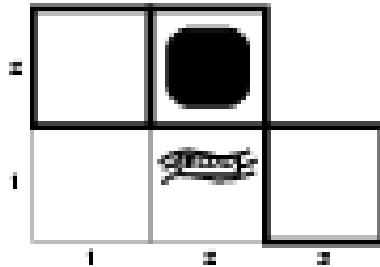
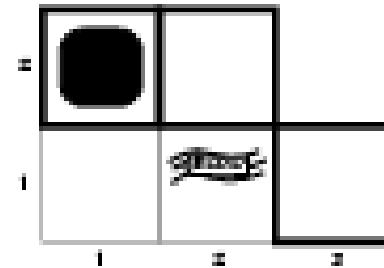
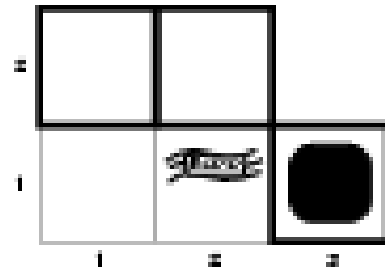
Situation after detecting
nothing in [1,1], moving
right, breeze in [2,1]

Consider possible models for
KB assuming only pits

3 Boolean choices $\Rightarrow$ 8
possible models



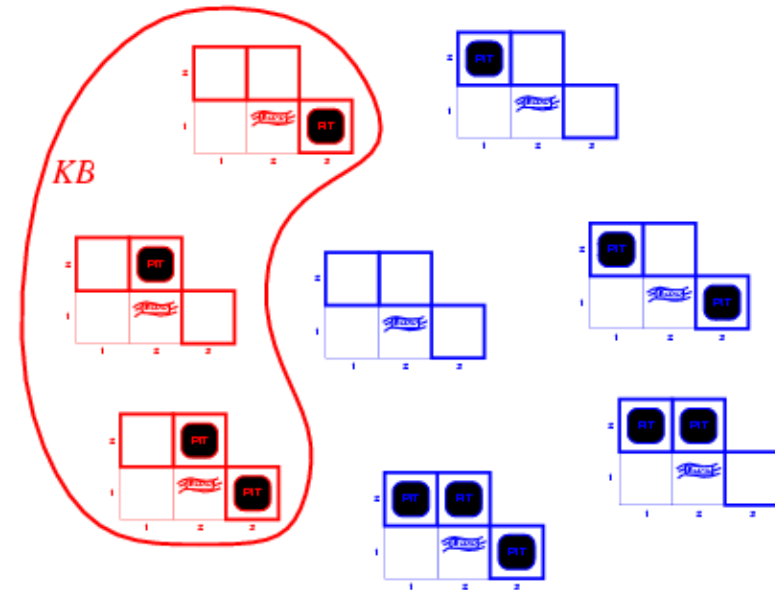
Wumpus models



Wumpus models

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
OK			
1,1	2,1	3,1	4,1
A			
OK	OK		

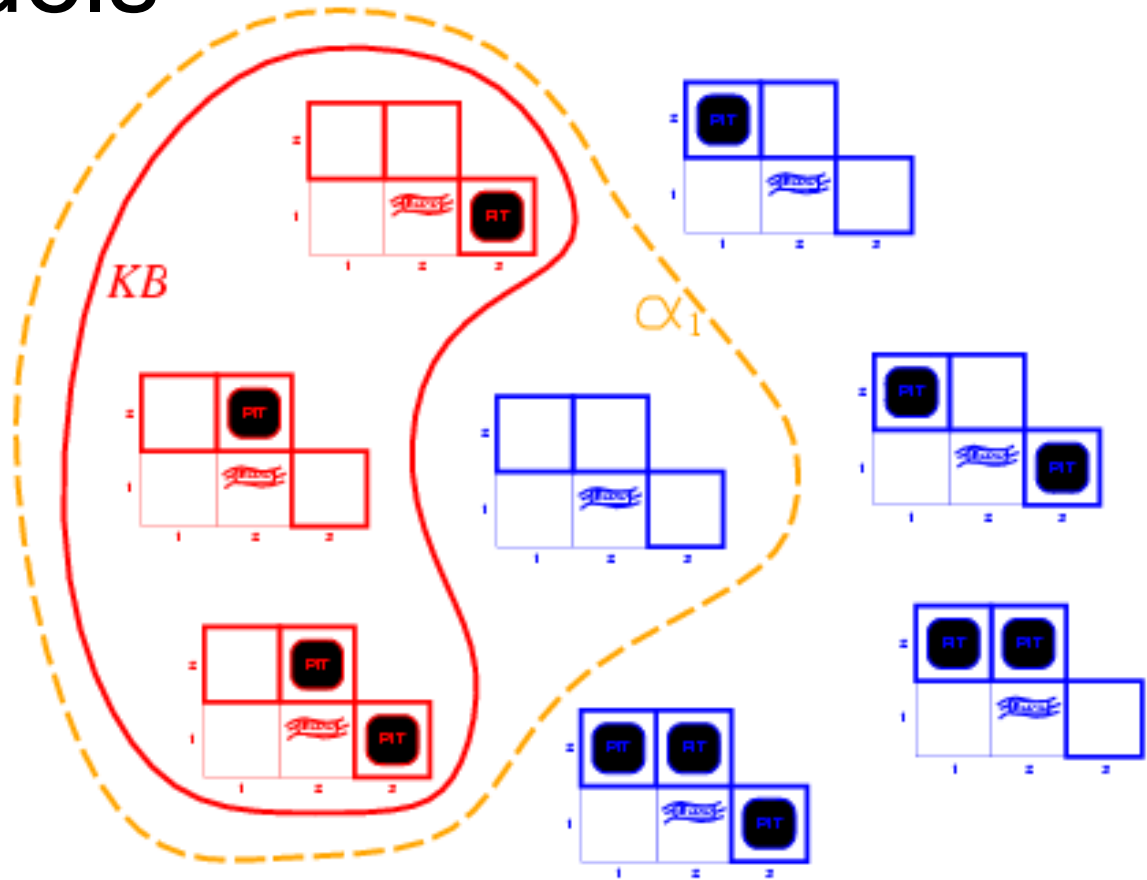
A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus



[None, None, None, None, None]

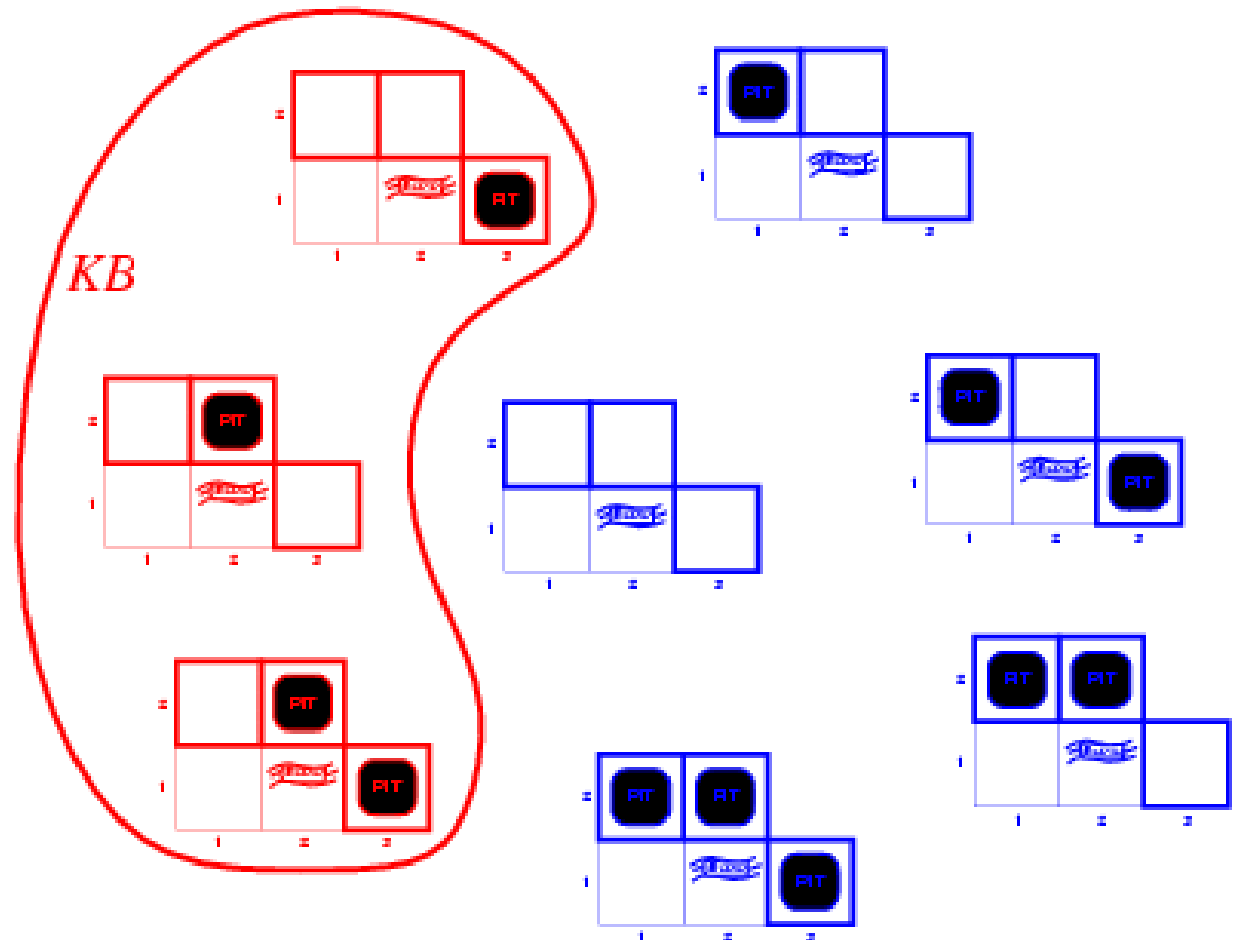
$KB = \text{Wumpus-world rules} + \text{observations}$

Wumpus models



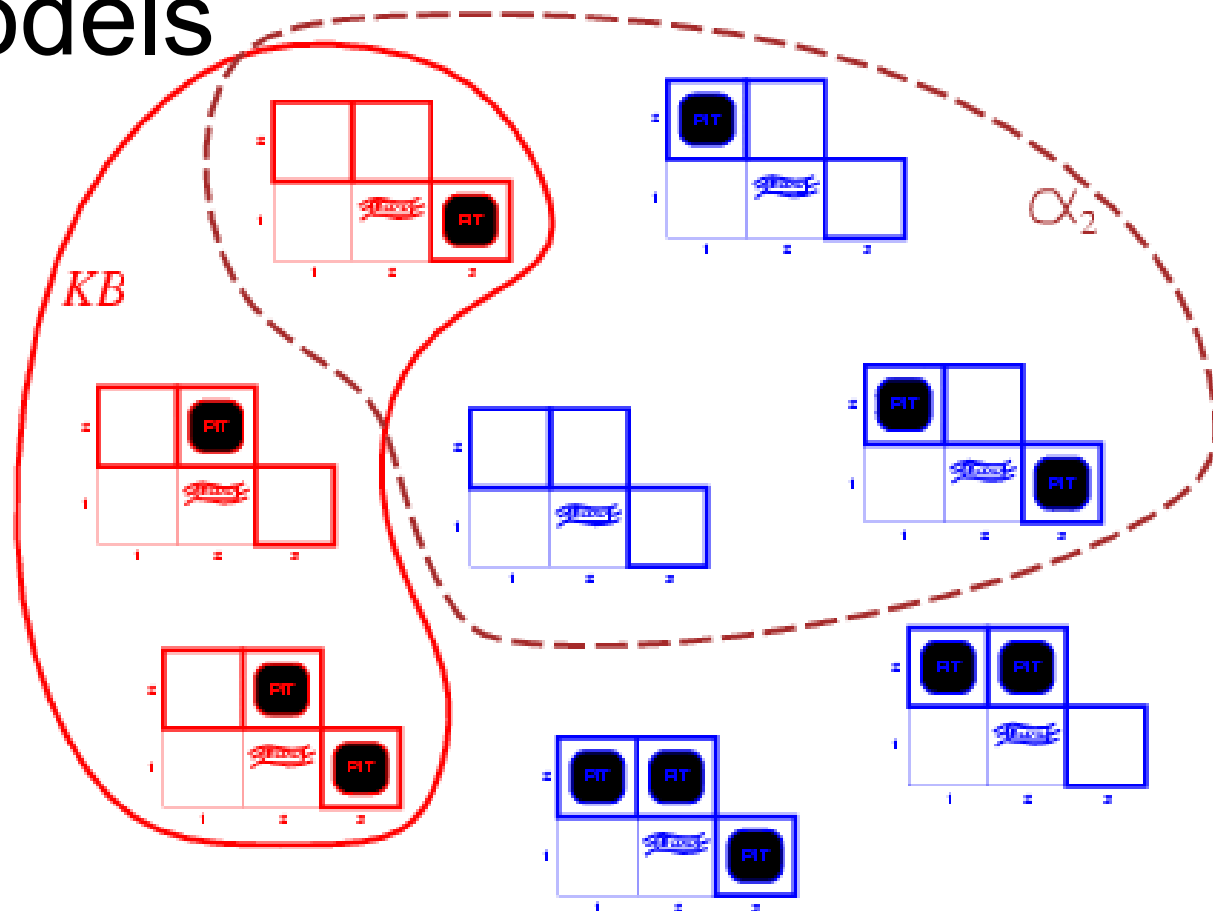
- KB = Wumpus-world rules + observations
- $\alpha_1 = "[1,2] \text{ is safe}"$, $KB \models \alpha_1$, proved by **model checking**

Wumpus models



- KB = Wumpus-world rules + observations

Wumpus models



- KB = Wumpus-world rules + observations
- α_2 = "[2,2] is safe", $KB \models \alpha_2$

Inference

- $KB \vdash_i \alpha$ = sentence α can be derived from KB by procedure i
- **Soundness**: i is sound if whenever $KB \vdash_i \alpha$, it is also true that $KB \models \alpha$
- **Completeness**: i is complete if whenever $KB \models \alpha$, it is also true that $KB \vdash_i \alpha$
- **Preview**: we will define a logic (first-order logic) which is expressive enough to say almost anything of interest, and for which there exists a sound and complete inference procedure.
- That is, the procedure will answer any question whose answer follows from what is known by the KB .

Propositional logic: Syntax

- Propositional logic is the simplest logic – illustrates basic ideas
- The proposition symbols P_1, P_2 etc are sentences
 - If S is a sentence, $\neg S$ is a sentence (**negation**)
 - If S_1 and S_2 are sentences, $S_1 \wedge S_2$ is a sentence (**conjunction**)
 - If S_1 and S_2 are sentences, $S_1 \vee S_2$ is a sentence (**disjunction**)
 - If S_1 and S_2 are sentences, $S_1 \Rightarrow S_2$ is a sentence (**implication**)
 - If S_1 and S_2 are sentences, $S_1 \Leftrightarrow S_2$ is a sentence (**biconditional**)

Propositional logic: Semantics

Each model specifies true/false for each proposition symbol

E.g. $P_{1,2}$ false $P_{2,2}$ true $P_{3,1}$ false

With these symbols, 8 possible models, can be enumerated automatically.

Rules for evaluating truth with respect to a model m :

$\neg S$	is true if	S is false	
$S_1 \wedge S_2$	is true if	S_1 is true and	S_2 is true
$S_1 \vee S_2$	is true if	S_1 is true or	S_2 is true
$S_1 \Rightarrow S_2$	is true if	S_1 is false or	S_2 is true
i.e.,	is false if	S_1 is true and	S_2 is false
$S_1 \Leftrightarrow S_2$	is true if	$S_1 \Rightarrow S_2$ is true and	$S_2 \Rightarrow S_1$ is true

Simple recursive process evaluates an arbitrary sentence, e.g.,

$$\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1}) = \text{true} \wedge (\text{true} \vee \text{false}) = \text{true} \wedge \text{true} = \text{true}$$

Truth tables for connectives

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

Wumpus world sentences

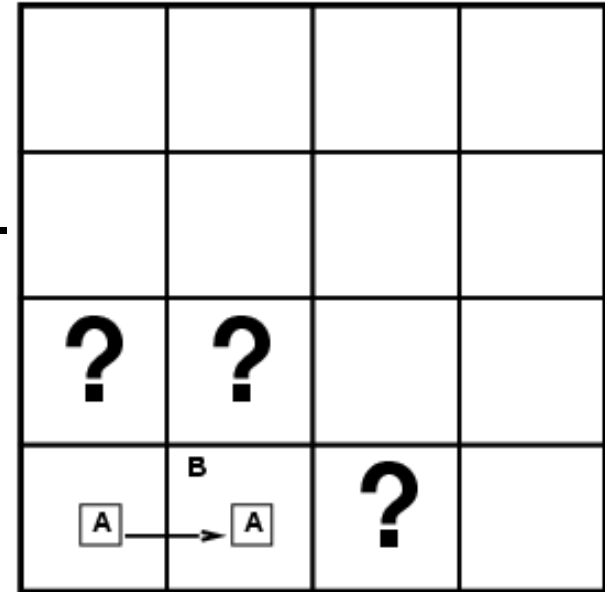
Let $P_{i,j}$ be true if there is a pit in $[i, j]$.

Let $B_{i,j}$ be true if there is a breeze in $[i, j]$.

$$\neg P_{1,1}$$

$$\neg B_{1,1}$$

$$B_{2,1}$$



- "Pits cause breezes in adjacent squares"

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

$$B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$$

Truth tables for inference

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	KB	α_1
false	false	false	false	false	false	false	false	true
false	false	false	false	false	false	true	false	true
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
false	true	false	false	false	false	false	false	true
false	true	false	false	false	false	true	<u>true</u>	<u>true</u>
false	true	false	false	false	true	false	<u>true</u>	<u>true</u>
false	true	false	false	false	true	true	<u>true</u>	<u>true</u>
false	true	false	false	true	false	false	false	true
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
true	true	true	true	true	true	true	false	false

Inference by enumeration

- Depth-first enumeration of all models is sound and complete

```
function TT-ENTAILS?(KB,  $\alpha$ ) returns true or false
```

```
  symbols  $\leftarrow$  a list of the proposition symbols in KB and  $\alpha$ 
```

```
  return TT-CHECK-ALL(KB,  $\alpha$ , symbols, [])
```

```
function TT-CHECK-ALL(KB,  $\alpha$ , symbols, model) returns true or false
```

```
  if EMPTY?(symbols) then
```

```
    if PL-TRUE?(KB, model) then return PL-TRUE?( $\alpha$ , model)
```

```
    else return true
```

```
  else do
```

```
    P  $\leftarrow$  FIRST(symbols); rest  $\leftarrow$  REST(symbols)
```

```
    return TT-CHECK-ALL(KB,  $\alpha$ , rest, EXTEND(P, true, model)) and
```

```
      TT-CHECK-ALL(KB,  $\alpha$ , rest, EXTEND(P, false, model))
```

- For n symbols, time complexity is $O(2^n)$, space complexity is $O(n)$

Logical equivalence

- Two sentences are **logically equivalent** iff true in same models: $\alpha \equiv \beta$ iff $\alpha \models \beta$ and $\beta \models \alpha$

$$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha) \quad \text{commutativity of } \wedge$$

$$(\alpha \vee \beta) \equiv (\beta \vee \alpha) \quad \text{commutativity of } \vee$$

$$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma)) \quad \text{associativity of } \wedge$$

$$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma)) \quad \text{associativity of } \vee$$

$$\neg(\neg\alpha) \equiv \alpha \quad \text{double-negation elimination}$$

$$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha) \quad \text{contraposition}$$

$$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta) \quad \text{implication elimination}$$

$$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) \quad \text{biconditional elimination}$$

$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta) \quad \text{de Morgan}$$

$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta) \quad \text{de Morgan}$$

$$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) \quad \text{distributivity of } \wedge \text{ over } \vee$$

$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) \quad \text{distributivity of } \vee \text{ over } \wedge$$

Validity and satisfiability

Logical Equivalence: $\alpha \equiv \beta$ iff $\alpha \models \beta$ and $\beta \models \alpha$

A sentence is **valid** if it is true in **all** models,
e.g., *True*, $A \vee \neg A$, $A \Rightarrow A$, $(A \wedge (A \Rightarrow B)) \Rightarrow B$

Validity is connected to inference via the **Deduction Theorem**:
 $KB \models \alpha$ if and only if $(KB \Rightarrow \alpha)$ is valid

A sentence is **satisfiable** if it is true in **some** model
e.g., $A \vee B$,

A sentence is **unsatisfiable** if it is true in **no** models
e.g., $A \wedge \neg A$

Satisfiability is connected to inference via the following:
 $KB \models \alpha$ if and only if $(KB \wedge \neg \alpha)$ is unsatisfiable

Proof methods

- Proof methods divide into (roughly) two kinds:
 - Application of inference rules
 - *Legitimate (sound) generation of new sentences from old*
 - **Proof** = a sequence of inference rule applications
Can use inference rules as operators in a standard search algorithm
 - *Typically require transformation of sentences into a normal form*
 - Model checking
 - *truth table enumeration (always exponential in n)*
 - *improved backtracking, e.g., Davis--Putnam-Logemann-Loveland (DPLL)*
 - *heuristic search in model space (sound but incomplete)*
e.g., min-conflicts-like hill-climbing algorithms

Inference in Propositional Logic

$$R_1 : \neg P_{1,1}$$

$$R_2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

$$R_3 : B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$$

$$R_4 : \neg B_{1,1}$$

$$R_5 : B_{2,1}$$

$$R_6 : (B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

$$R_7 : ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}) \quad \text{(a): moving back to [1,2] from [2,1] stench and no breeze:}$$

$$R_8 : (\neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1}))$$

$$R_9 : \neg(P_{1,2} \vee P_{2,1})$$

$$R_{10} : \neg P_{1,2} \wedge \neg P_{2,1}$$

similar prior inference:
biconditional elimination on R_3
and Modus Ponens on R_5 :
resolution on R_{13} and R_{15} :
resolution on R_1 and R_{16} :

1,4	2,4	3,4	4,4
1,3 W?	2,3	3,3	4,3
1,2 A S OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P?	4,1

(a)

A: Agent
B: Breeze
G: Gold
OK: safe
P: Pit
S: Stench
V: Visited
W: Wumpus

1,4	2,4 P?	3,4	4,4
1,3 W?	2,3 A S G B	3,3 P?	4,3
1,2 S V OK	2,2 V OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P?	4,1

(b)

$$R_{11} : \neg B_{1,2}$$

$$R_{12} : B_{1,2} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{1,3})$$

$$R_{13} : \neg P_{2,2}$$

$$R_{14} : \neg P_{1,3}$$

$$R_{15} : P_{1,1} \vee P_{2,2} \vee P_{3,1}$$

$$R_{16} : P_{1,1} \vee P_{3,1}$$

$$R_{17} : P_{3,1}$$

Resolution

Conjunctive Normal Form (CNF): conjunction of disjunctions of literals
clauses

E.g., $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

1. Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.

$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

2. Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $\neg \alpha \vee \beta$.

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

3. Move $\neg$ inwards using de Morgan's rules and double-negation:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$$

4. Apply distributivity law ($\wedge$ over $\vee$) and flatten:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

Resolution

Conjunctive Normal Form (CNF): conjunction of disjunctions of literals
clauses

E.g., $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$

- Unit resolution** inference rule (for CNF):

$$\frac{\ell_i \vee \dots \vee \ell_k, m_1}{\ell_i \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k}$$

where ℓ_i and m_j are complementary literals.

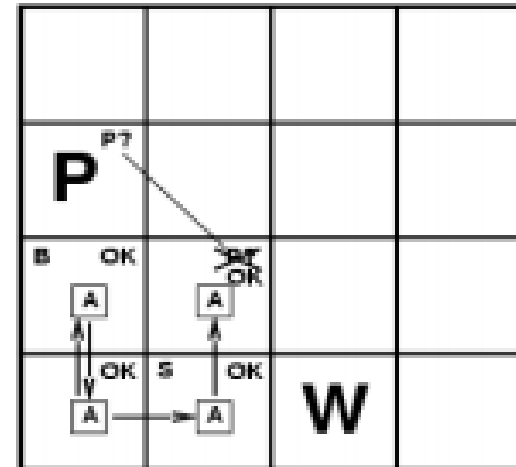
- Full Resolution** inference rule (for CNF):

$$\frac{\ell_i \vee \dots \vee \ell_k, m_1 \vee \dots \vee m_n}{\ell_i \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n}$$

where ℓ_i and m_j are complementary literals.

E.g., $\frac{P_{1,3} \vee P_{2,2}, \neg P_{2,2}}{P_{1,3}}$

- Resolution is sound and complete for propositional logic.



Resolution – Factoring

- Factoring is an inference rule that deals with eliminating redundancy within a single clause.
- The purpose of factoring is to simplify clauses that contain multiple instances of the same literal, possibly with different variables, by unifying those literals.

$$P(a,b) \vee P(x,y)$$

- If a and x , and b and y , can be unified (made identical by a substitution of variables), then the clause can be simplified.

$$(P(x) \vee Q(y) \vee P(z))$$

- In this clause, $P(x)$ and $P(z)$ can be unified by choosing x and z to be the same term. If we unify z to x , the clause becomes:

$$(P(x) \vee Q(y))$$

Resolution

- Factoring: Resulting clause should only contain one copy of a literal:
Elimination of multiple literal copies is called factoring (not necessary for set notation of clauses).

- Soundness of resolution inference rule:

$$\neg(l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k) \Rightarrow l_i$$

$$\neg m_j \Rightarrow (m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n)$$

$$\neg(l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k) \Rightarrow (m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n)$$

- Resolution is basis of a whole family of complete inference procedures:
 - Every complete search algorithm which uses resolution can derive any conclusion entailed by any knowledge base in PL.
- Resolution is **refutation complete**:
 - Given that A is true, It can be used to answer if $A \vee B$ is true but not to generate the consequence $A \vee B$

Propositional Resolution Propositional Resolution

- Resolution rule:

$$\frac{\alpha \vee \beta \quad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

Propositional Resolution Propositional Resolution

- Resolution rule:

$$\frac{\alpha \vee \beta \quad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

- Resolution refutation:

Propositional Resolution Propositional Resolution

- Resolution rule:

$$\frac{\alpha \vee \beta \quad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

- Resolution refutation:
 - Convert all sentences to CNF

Propositional Resolution Propositional Resolution

- Resolution rule:

$$\frac{\alpha \vee \beta \quad \neg \beta \vee \gamma}{\alpha \vee \gamma}$$

- Resolution refutation:
 - Convert all sentences to CNF
 - Negate the desired conclusion (converted to CNF)
 - Apply resolution rule until either
 - Derive false (a contradiction)
 - Can't apply any more

Propositional Resolution Propositional Resolution

- Resolution rule:

$$\frac{\begin{array}{l} \alpha \vee \beta \\ \neg\beta \vee \gamma \end{array}}{\alpha \vee \gamma}$$

- Resolution refutation:
 - Convert all sentences to CNF
 - Negate the desired conclusion (converted to CNF)
 - Apply resolution rule until either
 - Derive false (a contradiction)
 - Can't apply any more
- Resolution refutation is sound and complete
 - If we derive a contradiction, then the conclusion follows from the axioms
 - If we can't apply any more, then the conclusion cannot be proved from the axioms.

Resolution algorithm

- Proof by contradiction, i.e., show $KB \wedge \neg \alpha$ unsatisfiable

```
function PL-RESOLUTION( $KB, \alpha$ ) returns true or false
     $clauses \leftarrow$  the set of clauses in the CNF representation of  $KB \wedge \neg \alpha$ 
     $new \leftarrow \{ \}$ 
    loop do
        for each  $C_i, C_j$  in  $clauses$  do
             $resolvents \leftarrow$  PL-RESOLVE( $C_i, C_j$ )
            if  $resolvents$  contains the empty clause then return true
             $new \leftarrow new \cup resolvents$ 
        if  $new \subseteq clauses$  then return false
     $clauses \leftarrow clauses \cup new$ 
```

Resolution example

Say the agent is in [1,1], no breeze, so no pits can be in there

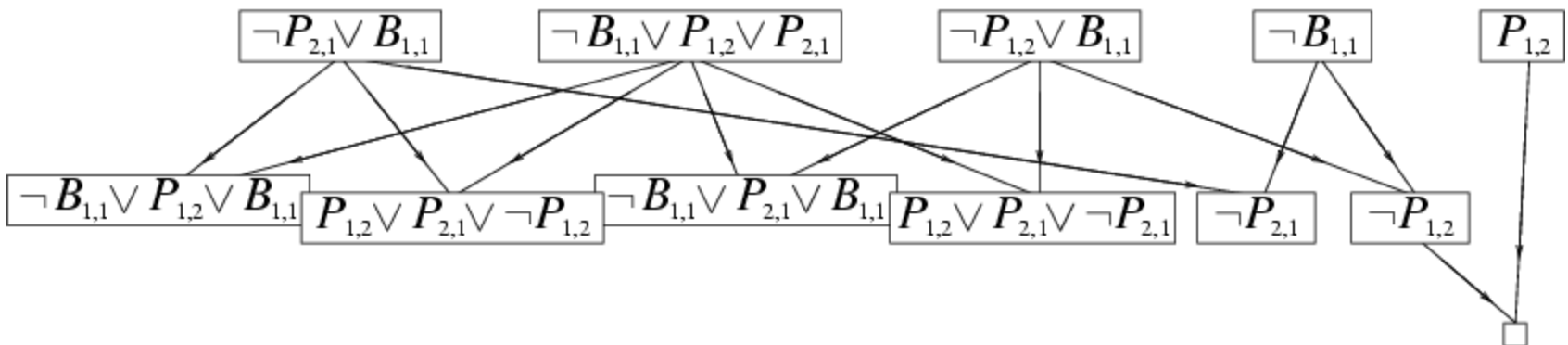
- $\alpha = \neg P_{1,2}$
- $KB = R_2 \wedge R_4$
- $KB = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1}$
- $KB \wedge \neg \alpha = \dots$

$$R_1 : \neg P_{1,1}$$

$$R_2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

$$R_3 : B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$$

$$R_4 : \neg B_{1,1}$$





University of Colorado
Colorado Springs



University of Colorado

Boulder | Colorado Springs | Denver | Anschutz Medical Campus

Lecture 8: Logical Agents Part II

Russell and Norvig: Chapter 7

CS-4820/5820

Tu/Th 12:15 PM-1:30 PM

Room: Centennial Hall 106

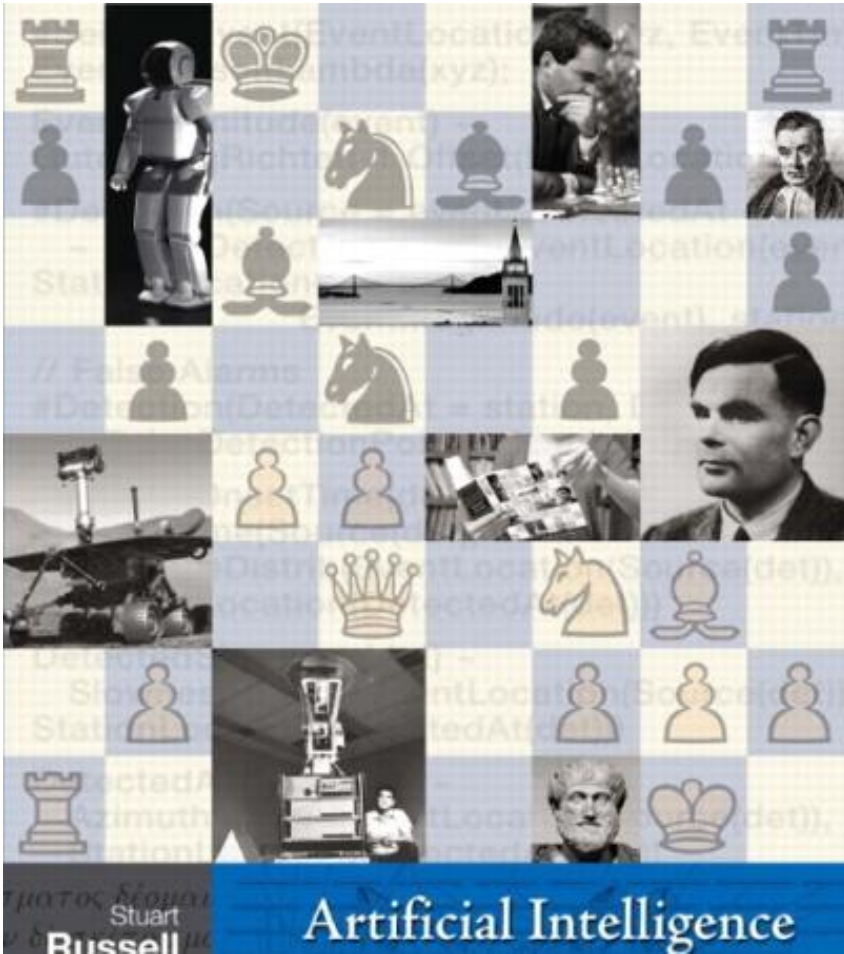
Instructor: Adham Atyabi

Office: Engineering 243

Office Hours: Mon 9:00 AM-14:00 PM.

Email: aatyabi@uccs.edu

Teaching Assistant: Ali Al Shami
(aalshami@uccs.edu)



Outline

- Knowledge-based agents
- Wumpus world
- Logic in general - models and entailment
- Propositional (Boolean) logic
- Equivalence, validity, satisfiability
- Inference rules and theorem proving
 - forward chaining
 - backward chaining
 - resolution

Resolution

Conjunctive Normal Form (CNF): conjunction of disjunctions of literals
clauses

E.g., $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})\beta$$

1. Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.

$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

2. Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$.

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

3. Move $\neg$ inwards using de Morgan's rules and double-negation:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \vee \neg P_{2,1}) \vee B_{1,1})$$

4. Apply distributivity law ($\wedge$ over $\vee$) and flatten:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

Resolution

Conjunctive Normal Form (CNF): conjunction of disjunctions of literals
clauses

E.g., $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$

- Unit resolution** inference rule (for CNF):

$$\frac{\ell_i \vee \dots \vee \ell_k, m_1}{\ell_i \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k}$$

where ℓ_i and m_j are complementary literals.

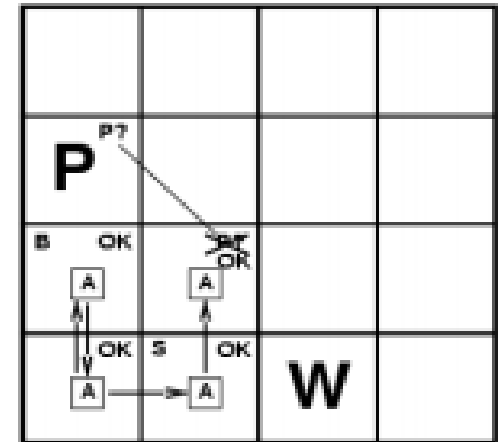
- Full Resolution** inference rule (for CNF):

$$\frac{\ell_i \vee \dots \vee \ell_k, m_1 \vee \dots \vee m_n}{\ell_i \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n}$$

where ℓ_i and m_j are complementary literals.

E.g., $\frac{P_{1,3} \vee P_{2,2}, \neg P_{2,2}}{P_{1,3}}$

- Resolution is sound and complete for propositional logic.



Forward and backward chaining

- **Horn Form** (restricted)

KB = **conjunction** of **Horn clauses**

- Horn clause (at most one literal is positive)
 - **proposition symbol (fact) or**
 - **(conjunction of symbols) $\Rightarrow$ symbol**
- E.g., $C \wedge (B \Rightarrow A) \wedge (C \wedge D \Rightarrow B)$
- $(\neg C \vee \neg B \vee A)$ becomes $(C \wedge B \Rightarrow A)$

- **Modus Ponens** (for Horn Form): complete for Horn KBs

$$\alpha_1, \dots, \alpha_n, \quad \alpha_1 \wedge \dots \wedge \alpha_n \Rightarrow \beta$$

$$\beta$$

- Can be used with **forward chaining** or **backward chaining**.
- These algorithms are very natural and run in **linear** time

Forward chaining algorithm

```
function PL-FC-ENTAILS?(KB, q) returns true or false
  local variables: count, a table, indexed by clause, initially the number of premises
                  inferred, a table, indexed by symbol, each entry initially false
                  agenda, a list of symbols, initially the symbols known to be true

  while agenda is not empty do
    p ← POP(agenda)
    unless inferred[p] do
      inferred[p] ← true
      for each Horn clause c in whose premise p appears do
        decrement count[c]
        if count[c] = 0 then do
          if HEAD[c] = q then return true
          PUSH(HEAD[c], agenda)

  return false
```

- Forward chaining is sound and complete for Horn KB

Forward chaining

- Idea: fire any rule whose premises are satisfied in the *KB*,
 - add its conclusion to the *KB*, until query is found

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

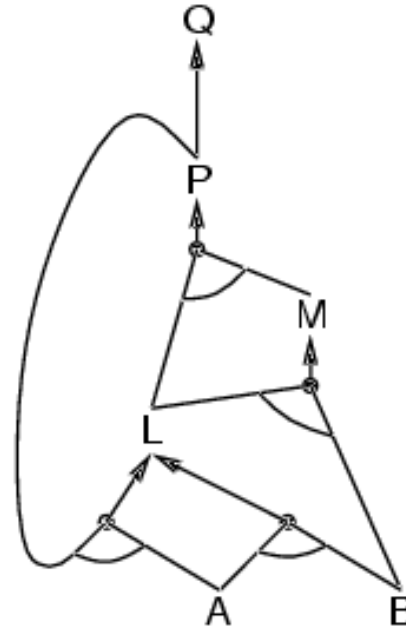
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Forward chaining example

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

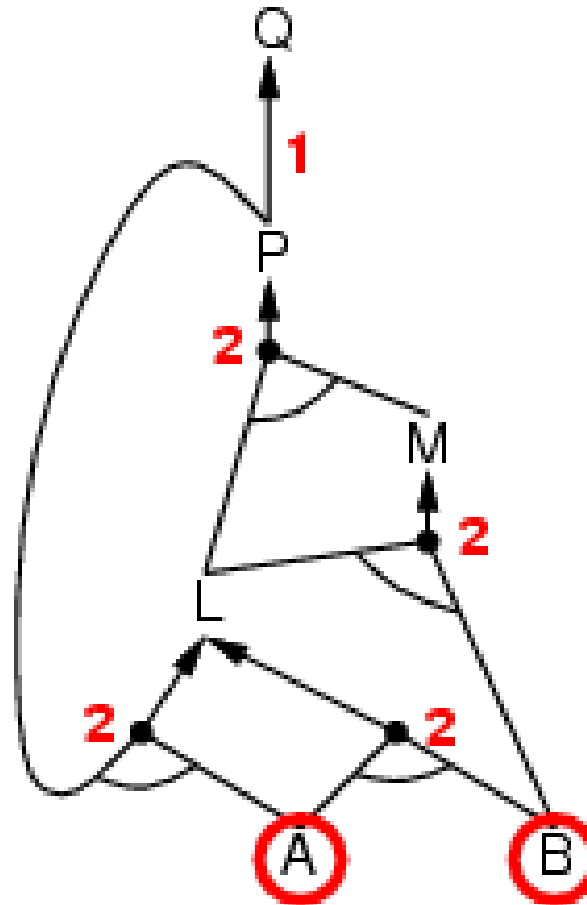
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

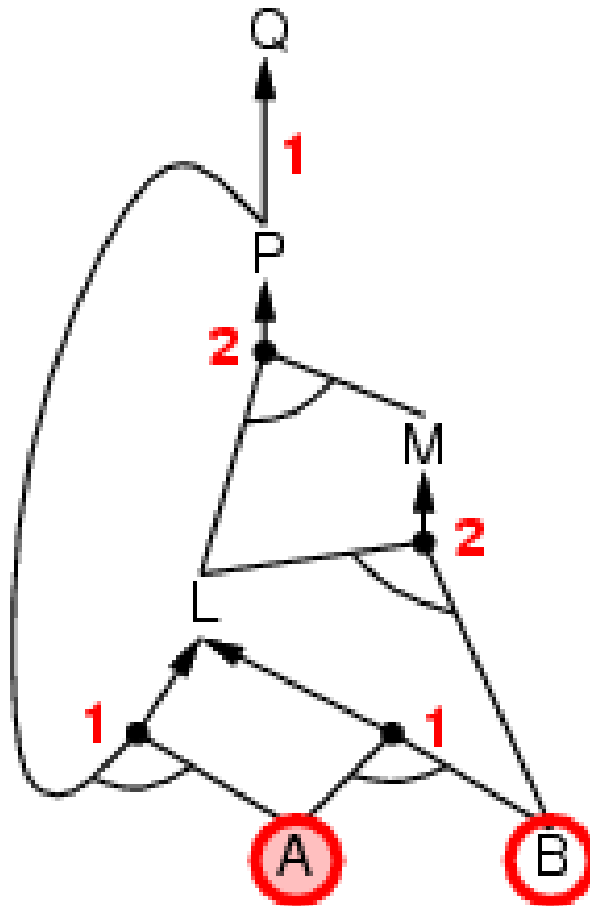
A

B



Forward chaining example

$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B



Forward chaining example

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

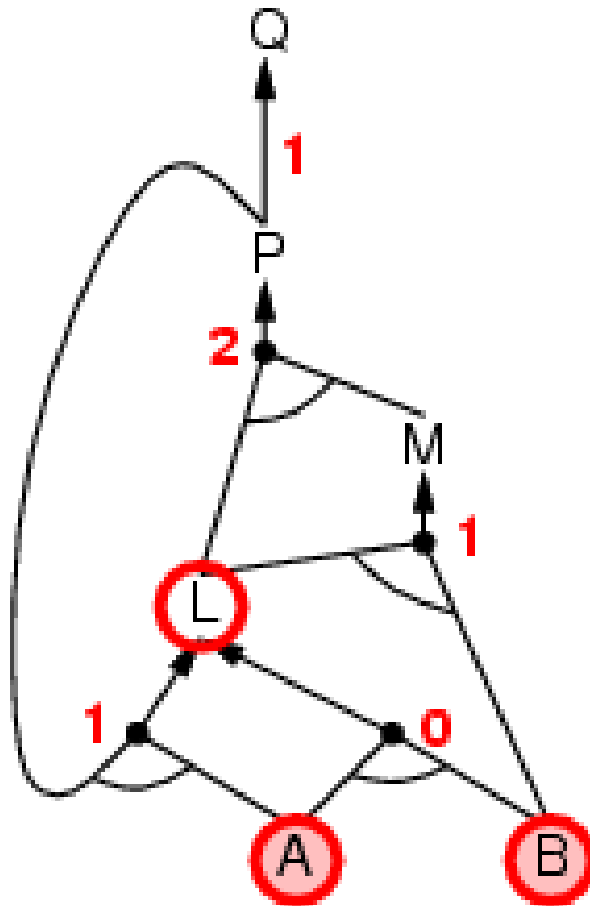
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

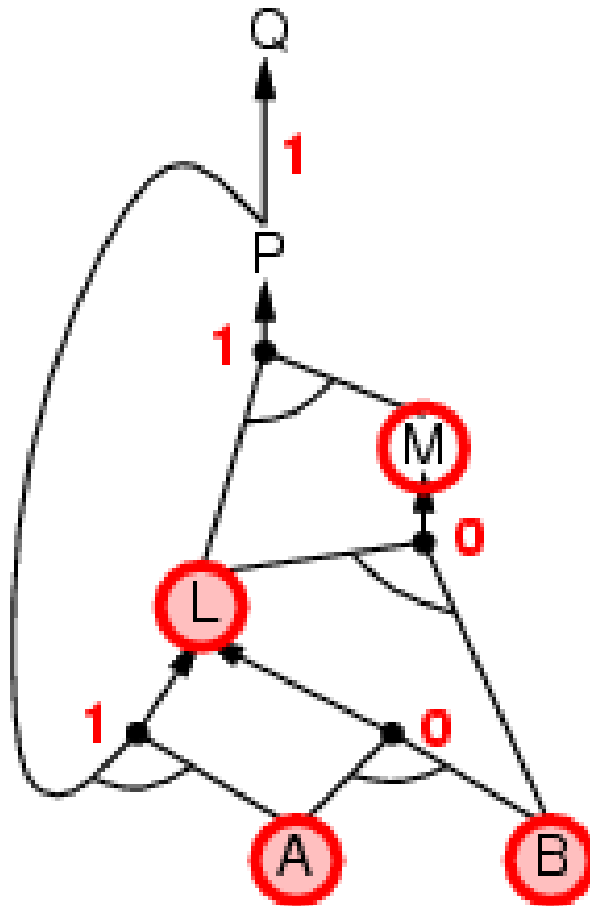
A

B



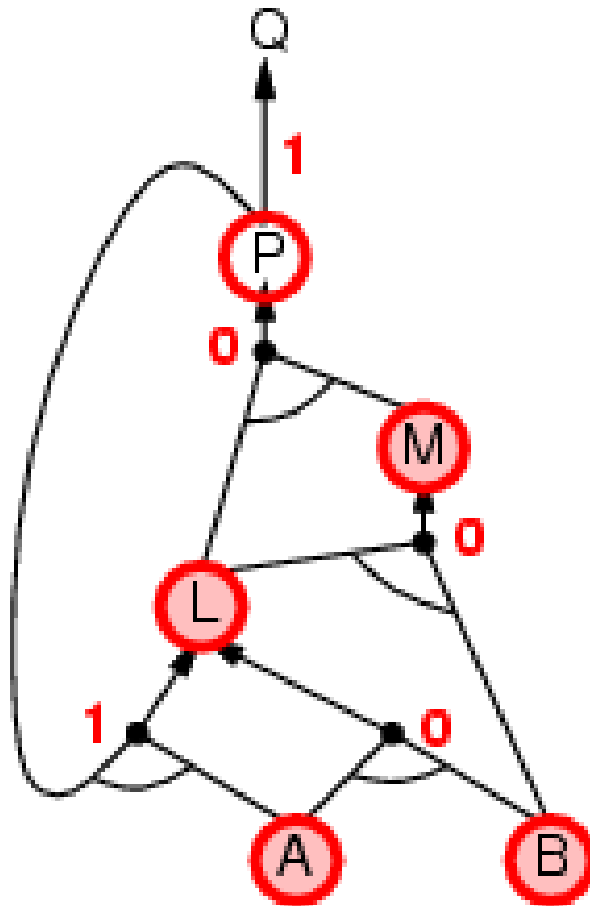
Forward chaining example

$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B



Forward chaining example

$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B



Forward chaining example

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

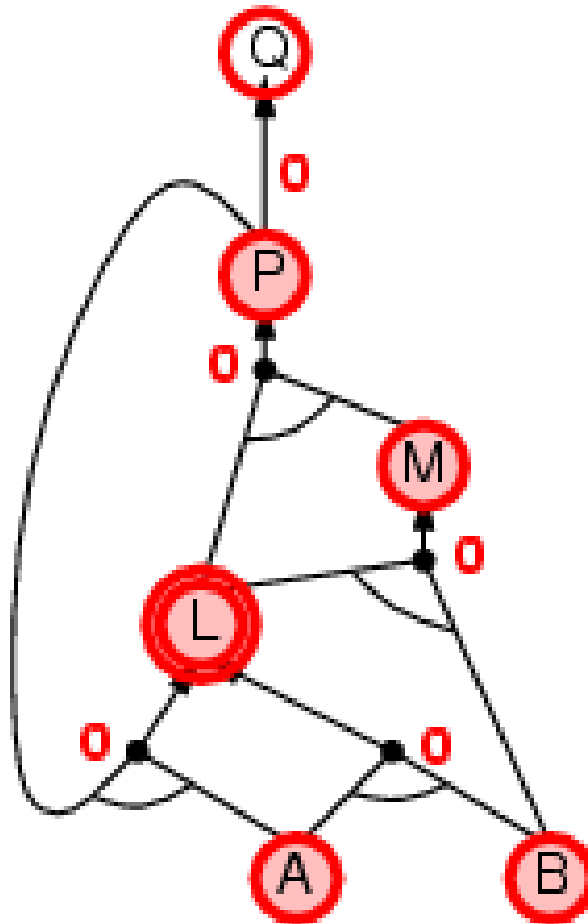
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Forward chaining example

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

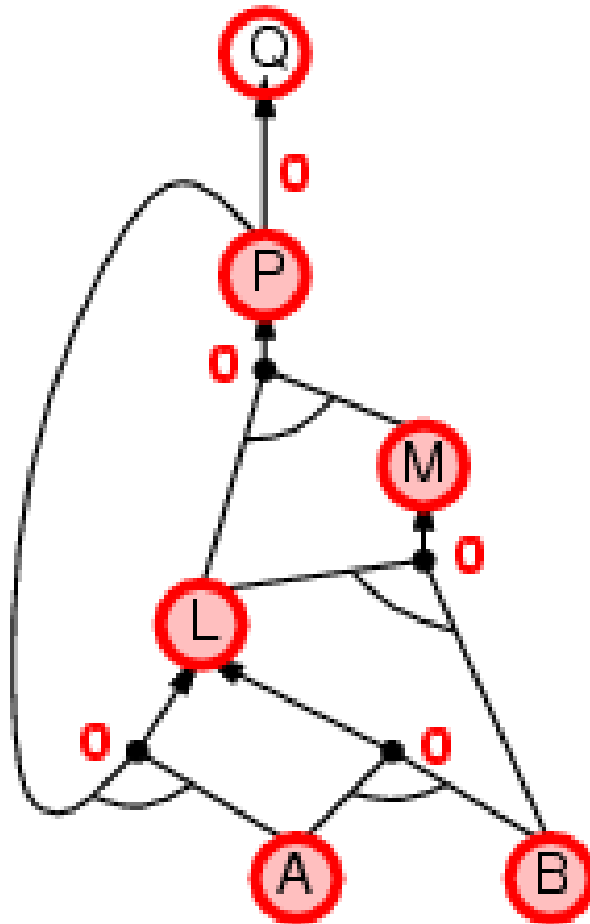
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Forward chaining example

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

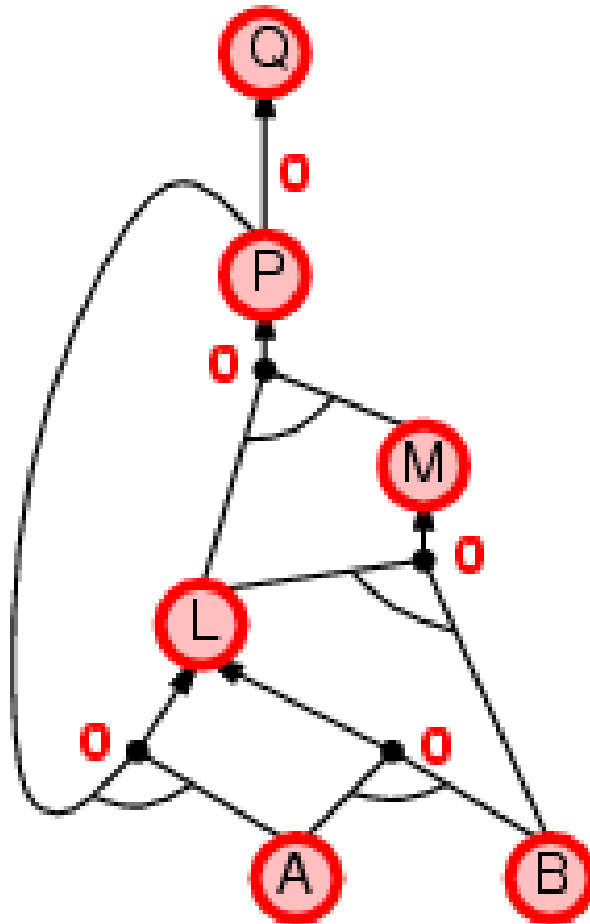
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Proof of completeness

- FC derives every atomic sentence that is entailed by KB
 1. FC reaches a **fixed point** where no new atomic sentences are derived
 2. Consider the final state as a model m , assigning true/false to symbols
 3. Every clause in the original KB is true in m
$$\mathbf{a_1 \wedge \dots \wedge a_k \Rightarrow b}$$
 4. Hence m is a model of KB
 5. If $KB \models q$, q is true in **every** model of KB , including m

Backward chaining

Idea: work backwards from the query q :

to prove q by BC,

check if q is known already, or

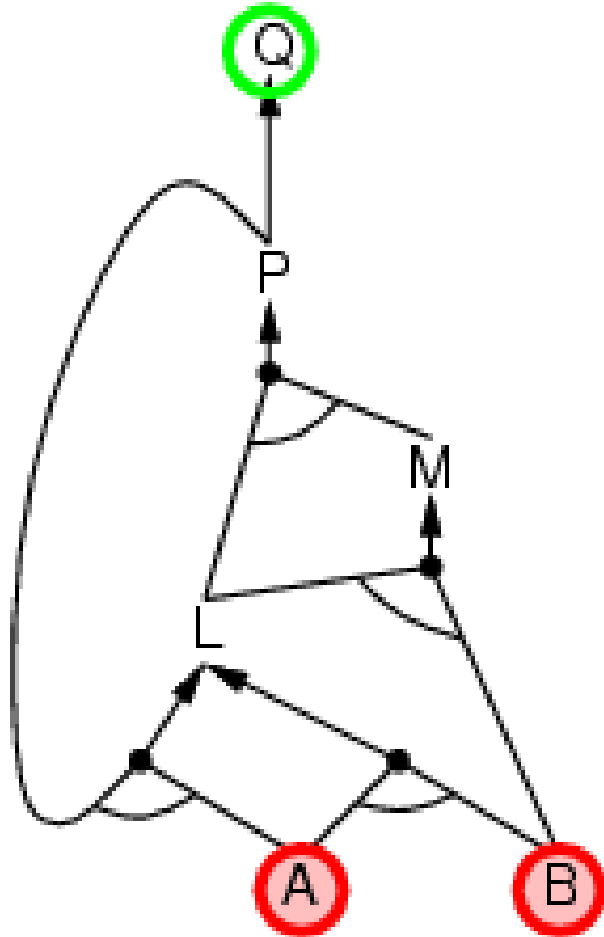
prove by BC all premises of some rule concluding q

Avoid loops: check if new subgoal is already on the goal stack

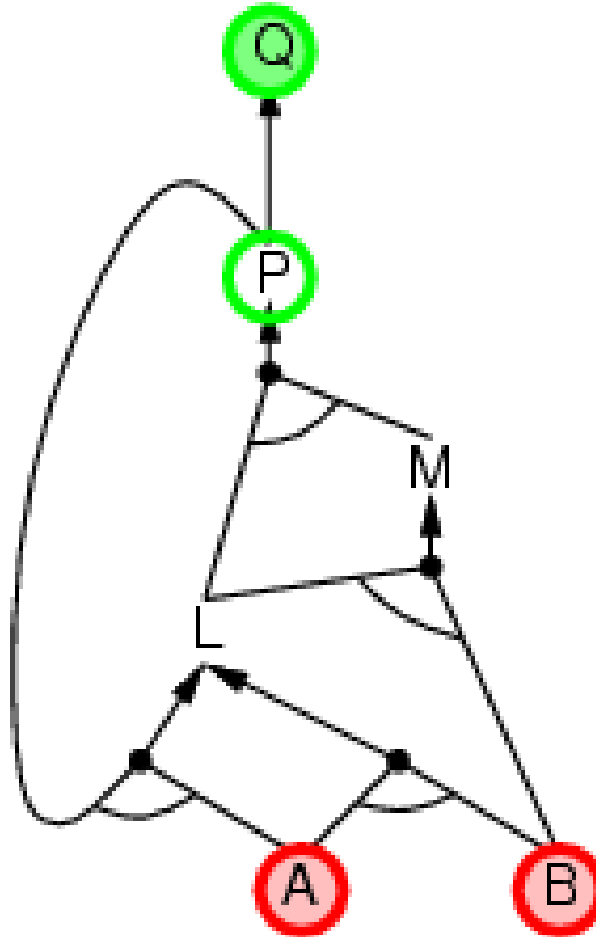
Avoid repeated work: check if new subgoal

1. has already been proved true, or
2. has already failed

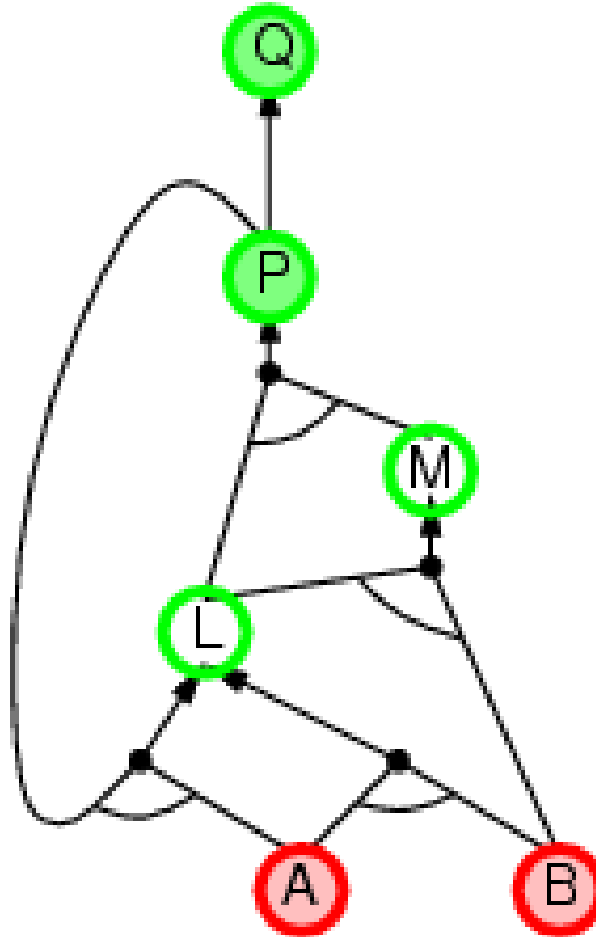
Backward chaining example



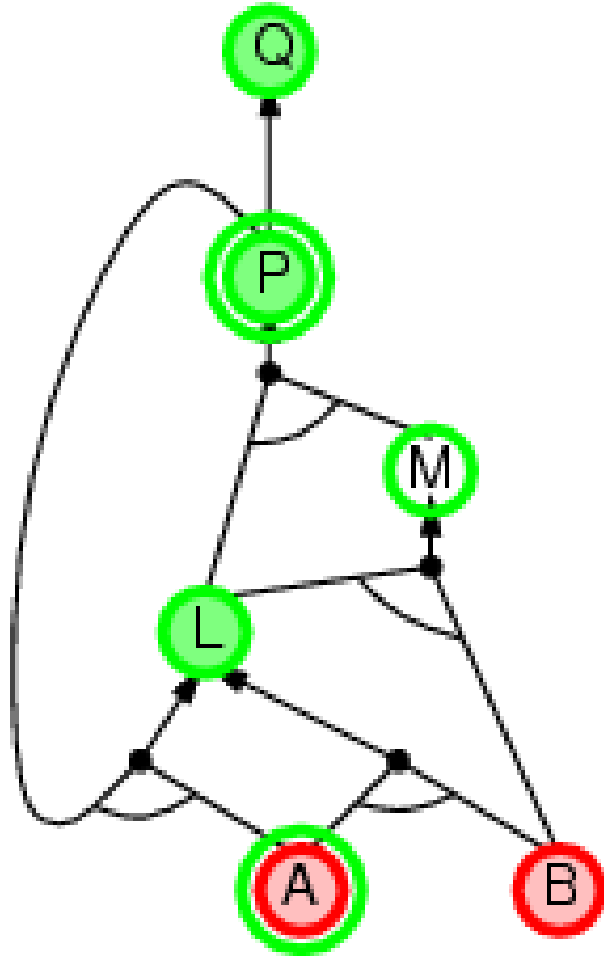
Backward chaining example



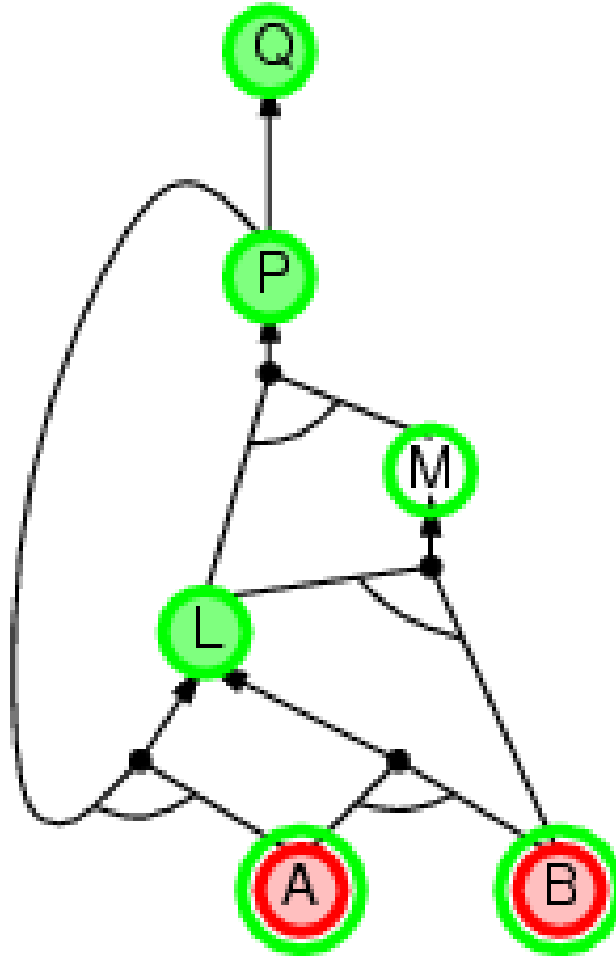
Backward chaining example



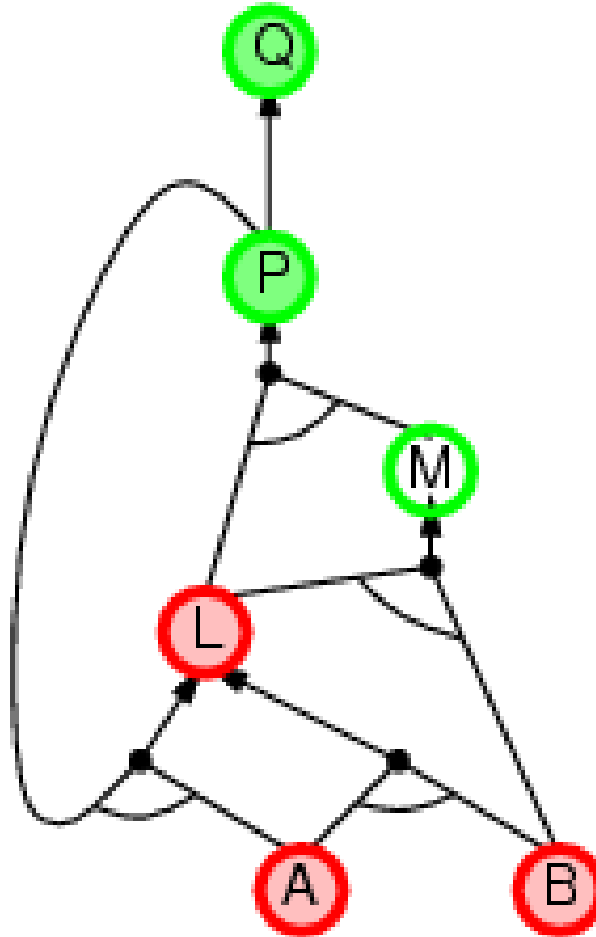
Backward chaining example



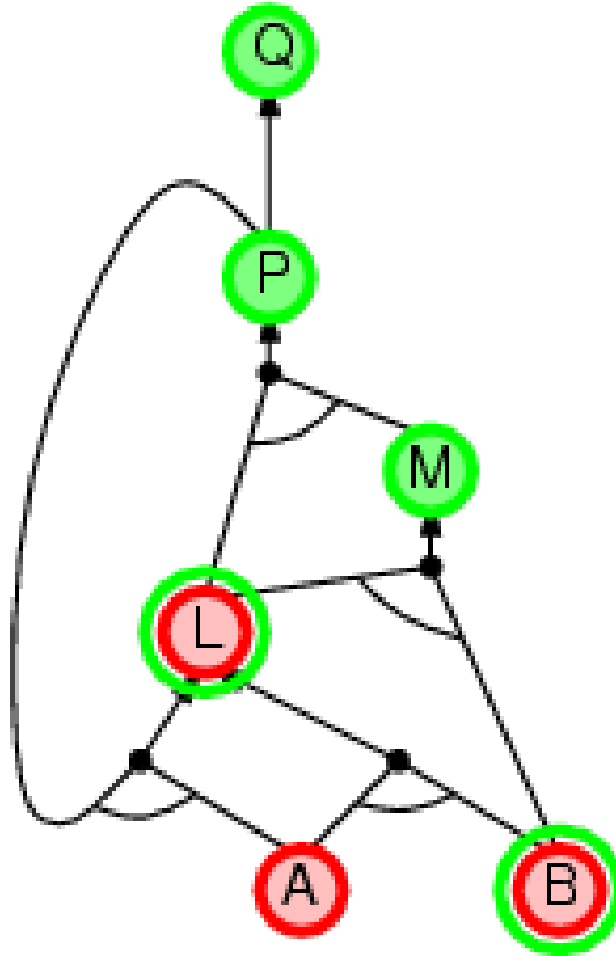
Backward chaining example



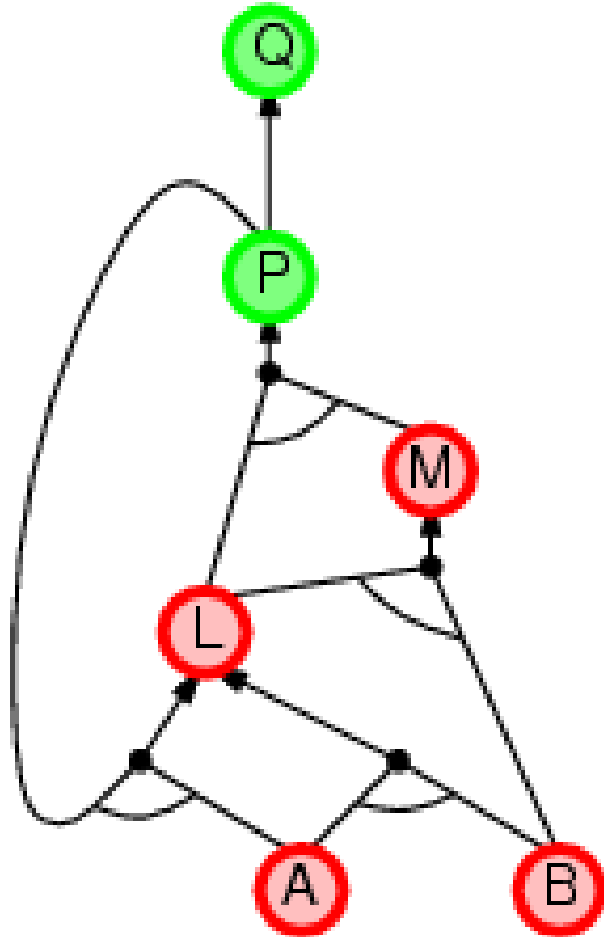
Backward chaining example



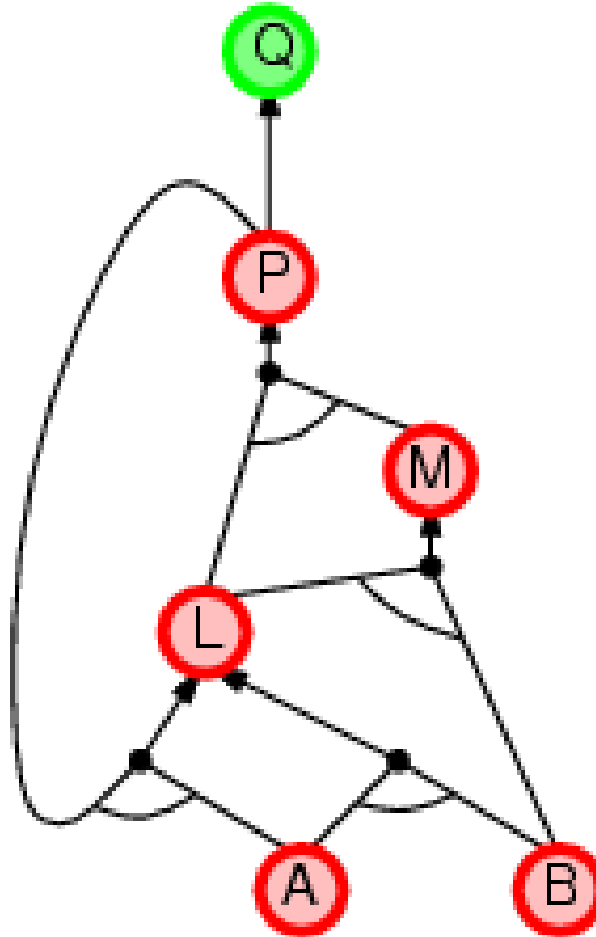
Backward chaining example



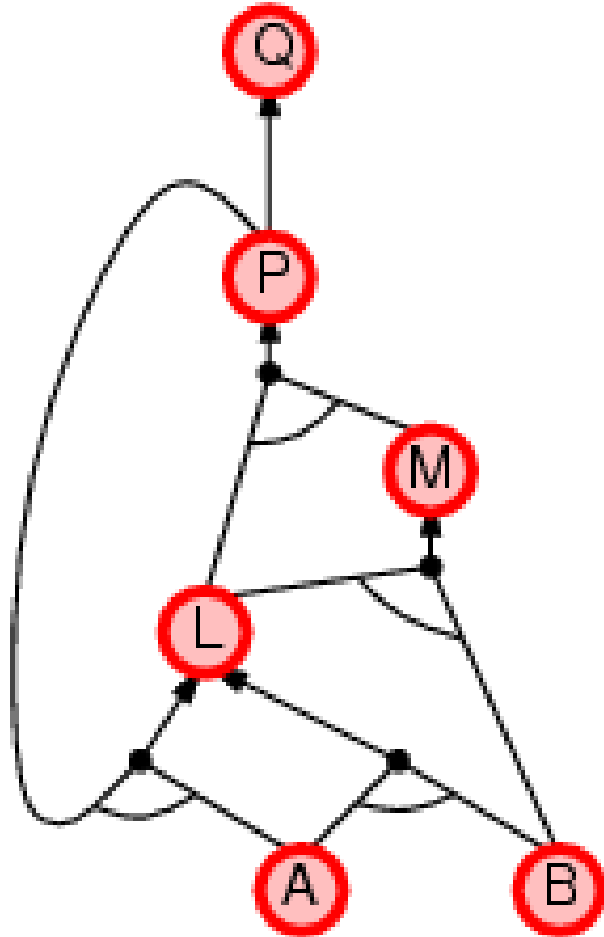
Backward chaining example



Backward chaining example



Backward chaining example



Forward vs. backward chaining

- FC is **data-driven**, automatic, unconscious processing,
 - e.g., object recognition, routine decisions
- FC may do lots of work that is irrelevant to the goal
- BC is **goal-driven**, appropriate for problem-solving,
 - e.g., Where are my keys? How do I get into a PhD program?
Where is home? What is the result of equation x ?
- Complexity of BC can be **much less** than linear in size of KB

Efficient propositional inference

Two families of efficient algorithms for propositional inference:

Complete backtracking search algorithms

- DPLL algorithm (Davis, Putnam, Logemann, Loveland)
- Incomplete local search algorithms
 - WalkSAT algorithm

The DPLL algorithm

Determine if an input propositional logic sentence (in CNF) is satisfiable.

Improvements over truth table enumeration:

1. Early termination
***A clause is true if any literal is true.
A sentence is false if any clause is false.***
2. Pure symbol heuristic
***Pure symbol: always appears with the same "sign" in all clauses.
e.g., In the three clauses $(A \vee \neg B)$, $(\neg B \vee \neg C)$, $(\neg C \vee A)$, A and B are pure, C is impure.
Make a pure symbol literal true.***
3. Unit clause heuristic
***Unit clause: only one literal in the clause
The only literal in a unit clause must be true.***

The DPLL algorithm

function DPLL-SATISFIABLE?(*s*) **returns** *true* or *false*

inputs: *s*, a sentence in propositional logic

clauses $\leftarrow$ the set of clauses in the CNF representation of *s*

symbols $\leftarrow$ a list of the proposition symbols in *s*

return DPLL(*clauses*, *symbols*, [])

function DPLL(*clauses*, *symbols*, *model*) **returns** *true* or *false*

if every clause in *clauses* is true in *model* **then return** *true*

if some clause in *clauses* is false in *model* **then return** *false*

P, *value* $\leftarrow$ FIND-PURE-SYMBOL(*symbols*, *clauses*, *model*)

if *P* is non-null **then return** DPLL(*clauses*, *symbols* - *P*, [*P* = *value* | *model*])

P, *value* $\leftarrow$ FIND-UNIT-CLAUSE(*clauses*, *model*)

if *P* is non-null **then return** DPLL(*clauses*, *symbols* - *P*, [*P* = *value* | *model*])

P $\leftarrow$ FIRST(*symbols*); *rest* $\leftarrow$ REST(*symbols*)

return DPLL(*clauses*, *rest*, [*P* = *true* | *model*]) **or**

DPLL(*clauses*, *rest*, [*P* = *false* | *model*])

The WalkSAT algorithm

- Incomplete, local search algorithm
- Evaluation function: The min-conflict heuristic of minimizing the number of unsatisfied clauses
- Balance between greediness and randomness